

Usuário:

No projeto frontend Brasileiro com Streamlit, tenho form_manut_rodada_criar_rodada.py:

```
import streamlit as st

import pandas as pd

from _services.estadio_service import EstadioService

from _services.clube_service import ClubeService

from _services.rodada_service import RodadaService

from models.rodada_dto import UltimaRodadaResponseSchema, CriarRodadaSchema

from pydantic import ValidationError

from guards.auth_guard import auth_guard

import requests

from helpers.errorHelper import message_error

from datetime import datetime, time

from typing import List

auth_guard()
```

Este trecho de código tem o objetivo de mostrar, por exemplo, a mensagem na inclusão "message": "Transação incluída com sucesso".

porque ao realizar a inclusão com sucesso, colocar a mensagem em session_state e executar o comando "st.rerun()".

Este comando recarrega a página e como a variável "toast" esta carregar, então vai aparecer no canto superior direito a mensagem

```

toast = st.session_state.pop("toast", None)

if toast:

    st.toast(toast["message"], icon=toast["icon"])

st.set_page_config(

    page_title="Criação de Rodada",

    page_icon="",

    layout="wide",

    initial_sidebar_state="auto",

)

st.caption("Cadastrar os 10 jogos da rodada, com data, hora, estádio e clubes mandante e
visitante.")

```

1. Services

```

_estadioService = EstadioService()

_clubeService = ClubeService()

_rodadaService = RodadaService()

_rodadaListaService = RodadaService()

_ultimaRodadaService = RodadaService()

param_serie, param_ano, param_rodada = st.columns([1, 1, 1])

with param_serie:

    serie = st.selectbox("Série", ["A", "B"])

with param_ano:

    anos = list(range(datetime.now().year, datetime.now().year + 1))

    ano = st.selectbox("Ano", anos, index=anos.index(2026))

```

2. Detectar primeira carga ou troca de série

```
first_load = "serie_anterior" not in st.session_state

serie_changed = st.session_state.get("serie_anterior") != serie

rodada_recem_incluida = st.session_state.pop("rodada_incluida", False)

st.session_state["serie_anterior"] = serie
```

4. Buscar a última rodada com a série/ano SELECIONADOS

```
ultima_rodada_cadastrada = UltimaRodadaResponseSchema(

ultima_rodada_cadastrada=0,

proxima_rodada=0

)

try:

response = _ultimaRodadaService._list_ultima_rodada(serie=serie,ano=ano)

ultima_rodada_cadastrada = response if response else UltimaRodadaResponseSchema(

ultima_rodada_cadastrada=0,

proxima_rodada=0

)

except ValidationError as e:

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

status = e.response.status_code if e.response is not None else "?"

body = e.response.text if e.response is not None else "sem resposta"

st.error(f'API UltimaRodadaService retornou {status}: {body}')

except Exception as e:
```

```
st.error(f'Ocorreu um erro inesperado: {str(e)}')
```

3. Se primeira carga ou troca de série, forçar number_input para a última rodada

```
if first_load or serie_changed:
```

```
    rodada_inicial = ultima_rodada_cadastrada.ultima_rodada_cadastrada
```

```
    if rodada_inicial > 0:
```

```
        st.session_state["numero_rodada"] = rodada_inicial
```

```
    else:
```

```
        st.session_state["numero_rodada"] = 1
```

4. Renderizar o number_input com key (pega o valor de session_state)

```
with param_rodada:
```

```
    numero_rodada = st.number_input(
```

```
        "Rodada",
```

```
        min_value=1,
```

```
        max_value=38,
```

```
        key="numero_rodada"
```

```
    )
```

5. Buscar jogos da rodada selecionada

```
rows_jogos_da_rodada = []
```

```
try:
```

```
    response = _rodadaListaService.list(serie=serie, ano=ano, rodada=numero_rodada,  
    carrega_jogos=False)
```

```
    rows_jogos_da_rodada = [x.model_dump() for x in response]
```

```

except ValidationError as e:

    for err in e.errors():

        st.error(err["msg"])

except requests.exceptions.HTTPError as e:

    status = e.response.status_code if e.response is not None else "?"

    if status == 404:

        rows_jogos_da_rodada = []

        st.error("Rodada informada não esta cadastrada.")

        st.stop

    else:

        body = e.response.text if e.response is not None else "sem resposta"

        st.error(f"API rodadaListaService.list retornou {status}: {body}")

except Exception as e:

    st.error(f'Ocorreu um erro inesperado rodadaListaService.list: {str(e)}")

```

6. Buscar estádios e clubes (continua igual)

```

rows_clube = []

rows_estadio = []

try:

    response = _estadioService.list(nome_estadio="", nome_cidade="", uf="")

    rows_estadio = [x.model_dump() for x in response]

except ValidationError as e:

    for err in e.errors():

        st.error(err["msg"])

except requests.exceptions.HTTPError as e:

```

```

status = e.response.status_code if e.response is not None else "?"

body = e.response.text if e.response is not None else "sem resposta"

st.error(f"API estadioService.list retornou {status}: {body}")

except Exception as e:

st.error(f"Ocorreu um erro inesperado estadioService.list: {str(e)}")

try:

response = _clubeService.list(serie=serie, nome="")

rows_clube = [x.model_dump() for x in response]

except ValidationError as e:

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

status = e.response.status_code if e.response is not None else "?"

body = e.response.text if e.response is not None else "sem resposta"

st.error(f"API clubeService.list retornou {status}: {body}")

except Exception as e:

st.error(f"Ocorreu um erro inesperado clubeService.list: {str(e)}")

estadio_opcoes = {

row["est_nome"]: row["est_id"]

for row in rows_estadio

if row.get("est_nome")

}

clube_opcoes = {

row["clu_nome"]: row["clu_sigla"]

for row in rows_clube

```

```
if row.get("clu_nome")

}

if not estadio_opcoes:

st.error("Nenhum estádio foi encontrado.")

st.stop()

if not clube_opcoes:

st.error("Nenhum clube foi encontrado.")

st.stop()

estadio_lista = [""] + list(estadio_opcoes.keys())

clube_lista = [""] + list(clube_opcoes.keys())

rodada_jogos_df = pd.DataFrame(rows_jogos_da_rodada)
```

Garantir os tipos corretos

```
if not rodada_jogos_df.empty:

rodada_jogos_df["rod_rodada"] = pd.to_numeric(

rodada_jogos_df["rod_rodada"],

errors="coerce"

)

rodada_jogos_df["rod_data"] = pd.to_datetime(

rodada_jogos_df["rod_data"],

errors="coerce"

)

rodada_jogos_df["est_id"] = pd.to_numeric(

rodada_jogos_df["est_id"],
```

```

errors="coerce"

)

rodada_jogos_df["rod_gols_mandante"] = pd.to_numeric(
rodada_jogos_df["rod_gols_mandante"],
errors="coerce"

)

rodada_jogos_df["rod_gols_visitante"] = pd.to_numeric(
rodada_jogos_df["rod_gols_visitante"],
errors="coerce"

)

rodada_jogos_df["display_mandante"] = (
rodada_jogos_df["clube_clu_sigla_mandante"]
+ " - "
+ rodada_jogos_df["clu_nome_mandante"]
)

rodada_jogos_df["display_visitante"] = (
rodada_jogos_df["clube_clu_sigla_visitante"]
+ " - "
+ rodada_jogos_df["clu_nome_visitante"]
)

```

```

st.write("COLUMNS:",
rodada_jogos_df.columns.tolist())

```

```

st.write("DTYPES:", rodada_jogos_df.dtypes)

```



```
for coluna in rodada_jogos_df.columns:
```

```
st.write(
```

```
f"COLUNA: {coluna}",
```

```
rodada_jogos_df[coluna].tolist()
```

```
)
```

```
edited = st.dataframe(
```

```
rodada_jogos_df,
```

```
key="RodadaJogos",
```

```
on_select="ignore",
```

```
height="auto",
```

```
selection_mode=["single-row"],
```

```
column_order=[
```

```
"rod_rodada",
```

```
"rod_data",
```

```
"est_nome",
```

```
"display_mandante",
```

```
"clu_link_escudo_mandante",
```

```
"rod_gols_mandante",
```

```
"rod_gols_visitante",
```

```
"clu_link_escudo_visitante",
```

```
"display_visitante",
```

```
],
```

```

column_config={

"rod_rodada": st.column_config.NumberColumn("Rodada", format="%d", alignment="center"),

"rod_data": st.column_config.DateColumn("Data", format="DD/MM/YYYY HH:mm",
alignment="center"),

"rod_sequencia": st.column_config.NumberColumn("Sequência", format="%d"),

"est_nome": st.column_config.TextColumn("Estádio"),

"display_mandante": st.column_config.TextColumn("Mandante"),

"clu_link_escudo_mandante": st.column_config.ImageColumn("Escudo"),

"rod_gols_mandante": st.column_config.NumberColumn("Gols Mandante", format="%d",
alignment="center"),

"rod_gols_visitante": st.column_config.NumberColumn("Gols Visitante", format="%d",
alignment="center"),

"display_visitante": st.column_config.TextColumn("Visitante"),

"clu_link_escudo_visitante": st.column_config.ImageColumn("Escudo"),

"rod_pontos_mandante": st.column_config.NumberColumn("Pontos Mandante", format="%d"),

"rod_pontos_visitante": st.column_config.NumberColumn("Pontos Visitante", format="%d"),

"rod_partida_finalizada": st.column_config.TextColumn("Partida Finalizada"),

"rod_calculou_classificacao": st.column_config.TextColumn("Classificação Calculada"),

},

hide_index=True

)

```

Vou criar outra tela, a placar_atualizar.py para atualizar os placares dos jogos da rodada que é semelhante a tela form_manut_rodada_criar_rodada.py

A diferença será na chamada do serviço:

Vou trocar este:

```

response = _rodadaListaService.list(serie=serie, ano=ano, rodada=numero_rodada,
carrega_jogos=False)

```

Por:

```
response = _rodadaListaService.list_preencher_placar(serie=serie, ano=ano,
rodada=numero_rodada, carrega_jogos=False)
```

As duas telas possuem lógicas semelhantes.

Como fazer para não ter redundância de código?

IA:

A forma mais direta de eliminar a redundância é **extrair a lógica comum para uma função parametrizada**, recebendo o método de serviço como callback. Assim, cada página só diferencia a chamada do serviço e mantém todo o resto compartilhado.

Estratégia: função compartilhada + callback de serviço

Crie um arquivo `components/rodada_component.py` (ou `shared/rodada_view.py`) com a lógica que é idêntica entre as duas telas:

```
```python
```

### `components/rodada_component.py`

```
import streamlit as st
```

```
import pandas as pd
```

```
from _services.estadio_service import EstadioService
```

```
from _services.clube_service import ClubeService
```

```
from _services.rodada_service import RodadaService
```

```
from models.rodada_dto import UltimaRodadaResponseSchema
```

```
from pydantic import ValidationError
```

```
import requests
```

```
from datetime import datetime
```

```
def render_rodada_view(
```

```
*,
```

```
titulo_pagina: str,
```

```
page_icon: str,
```

```
caption: str,
```

```
fetch_jogos_fn=None, # callable(serie, ano, rodada) -> list[model]
```

```
):
```

```
"""
```

Renderiza a tela de rodada (criação ou atualização de placar).

Parâmetro chave:

fetch\_jogos\_fn: função que busca os jogos da rodada.

Assinatura: (serie: str, ano: int, rodada: int) -> list[Model]

- Criar rodada: usa RodadaService().list(...)
- Atualizar placar: usa RodadaService().list\_preencher\_placar(...)

```
"""
```

```
st.set_page_config(
```

```
page_title=titulo_pagina,
```

```
page_icon=page_icon,
```

```
layout="wide",
```

```
initial_sidebar_state="auto",
```

```
)
```

```
st.caption(caption)
```

## --- Services ---

```
_estadioService = EstadioService()

_clubeService = ClubeService()

_ultimaRodadaService = RodadaService()
```

### --- Toast (mensagem de sucesso pós-rerun) ---

```
toast = st.session_state.pop("toast", None)

if toast:

 st.toast(toast["message"], icon=toast["icon"])
```

### --- Parâmetros: Série, Ano, Rodada ---

```
param_serie, param_ano, param_rodada = st.columns([1, 1, 1])

with param_serie:

 serie = st.selectbox("Série", ["A", "B"])

with param_ano:

 anos = list(range(datetime.now().year, datetime.now().year + 1))

 ano = st.selectbox("Ano", anos, index=anos.index(2026))
```

### --- Detectar primeira carga ou troca de série ---

```
first_load = "serie_anterior" not in st.session_state

serie_changed = st.session_state.get("serie_anterior") != serie

rodada_recem_incluida = st.session_state.pop("rodada_incluida", False)

st.session_state["serie_anterior"] = serie
```

### --- Buscar última rodada cadastrada ---

```
ultima_rodada_cadastrada = UltimaRodadaResponseSchema(
```

```

ultima_rodada_cadastrada=0,

proxima_rodada=0,

)

try:

response = _ultimaRodadaService._list_ultima_rodada(serie=serie,ano=ano)

ultima_rodada_cadastrada = response if response else ultima_rodada_cadastrada

except ValidationError as e:

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

status = e.response.status_code if e.response is not None else "?"

body = e.response.text if e.response is not None else "sem resposta"

st.error(f"API UltimaRodadaService retornou {status}: {body}")

except Exception as e:

st.error(f"Ocorreu um erro inesperado: {str(e)}")

```

## **--- Forçar number\_input para a última rodada na primeira carga ---**

```

if first_load or serie_changed:

rodada_inicial = ultima_rodada_cadastrada.ultima_rodada_cadastrada

st.session_state["numero_rodada"] = rodada_inicial if rodada_inicial > 0 else 1

with param_rodada:

numero_rodada = st.number_input(

"Rodada", min_value=1, max_value=38, key="numero_rodada"

)

```

## --- Buscar jogos da rodada (AQUI ESTÁ A DIFERENÇA) ---

```
rows_jogos_da_rodada = []

try:

 response = fetch_jogos_fn(serie=serie, ano=ano, rodada=numero_rodada)

 rows_jogos_da_rodada = [x.model_dump() for x in response] if response else []

except ValidationError as e:

 for err in e.errors():

 st.error(err["msg"])

except requests.exceptions.HTTPError as e:

 status = e.response.status_code if e.response is not None else "?"

 if status == 404:

 rows_jogos_da_rodada = []

 st.error("Rodada informada não está cadastrada.")

 st.stop()

 else:

 body = e.response.text if e.response is not None else "sem resposta"

 st.error(f"API retornou {status}: {body}")

except Exception as e:

 st.error(f'Ocorreu um erro inesperado: {str(e)}')
```

## --- Buscar estádios e clubes ---

```
rows_clube = []

rows_estadio = []

try:

 response = _estadioService.list(nome_estadio="", nome_cidade="", uf="")
```

```

rows_estadio = [x.model_dump() for x in response]

except (ValidationError, requests.exceptions.HTTPError, Exception) as e:

 _handle_service_error(e, "estadioService.list")

try:

 response = _clubeService.list(serie=serie, nome="")

 rows_clube = [x.model_dump() for x in response]

 except (ValidationError, requests.exceptions.HTTPError, Exception) as e:

 _handle_service_error(e, "clubeService.list")

 estadio_opcoes = {

 row["est_nome"]: row["est_id"]

 for row in rows_estadio

 if row.get("est_nome")

 }

 clube_opcoes = {

 row["clu_nome"]: row["clu_sigla"]

 for row in rows_clube

 if row.get("clu_nome")

 }

 if not estadio_opcoes:

 st.error("Nenhum estádio foi encontrado.")

 st.stop()

 if not clube_opcoes:

 st.error("Nenhum clube foi encontrado.")

 st.stop()

 estadio_lista = [""] + list(estadio_opcoes.keys())

```



```
clube_lista = [""] + list(clube_opcoes.keys())
```

## --- DataFrame dos jogos ---

```
rodada_jogos_df = pd.DataFrame(rows_jogos_da_rodada)
```

```
if not rodada_jogos_df.empty:
```

```
 rodada_jogos_df["rod_rodada"] = pd.to_numeric(
```

```
 rodada_jogos_df["rod_rodada"], errors="coerce"
```

```
)
```

```
 rodada_jogos_df["rod_data"] = pd.to_datetime(
```

```
 rodada_jogos_df["rod_data"], errors="coerce"
```

```
)
```

```
 rodada_jogos_df["est_id"] = pd.to_numeric(
```

```
 rodada_jogos_df["est_id"], errors="coerce"
```

```
)
```

```
 rodada_jogos_df["rod_gols_mandante"] = pd.to_numeric(
```

```
 rodada_jogos_df["rod_gols_mandante"], errors="coerce"
```

```
)
```

```
 rodada_jogos_df["rod_gols_visitante"] = pd.to_numeric(
```

```
 rodada_jogos_df["rod_gols_visitante"], errors="coerce"
```

```
)
```

```
 rodada_jogos_df["display_mandante"] = (
```

```
 rodada_jogos_df["clube_clu_sigla_mandante"]
```

```
 + " - "
```

```
 + rodada_jogos_df["clu_nome_mandante"]
```

```
)
```

```

rodada_jogos_df["display_visitante"] = (
rodada_jogos_df["clube_clu_sigla_visitante"]
+ " - "
+ rodada_jogos_df["clu_nome_visitante"]
)

edited = st.dataframe(
rodada_jogos_df,
key="RodadaJogos",
on_select="ignore",
height="auto",
selection_mode=["single-row"],
column_order=[
"rod_rodada", "rod_data", "est_nome",
"display_mandante", "clu_link_escudo_mandante",
"rod_gols_mandante", "rod_gols_visitante",
"clu_link_escudo_visitante", "display_visitante",
],
column_config={
"rod_rodada": st.column_config.NumberColumn("Rodada", format="%d", alignment="center"),
"rod_data": st.column_config.DateColumn("Data", format="DD/MM/YYYY HH:mm",
alignment="center"),
"est_nome": st.column_config.TextColumn("Estádio"),
"display_mandante": st.column_config.TextColumn("Mandante"),
"clu_link_escudo_mandante": st.column_config.ImageColumn("Escudo"),
"rod_gols_mandante": st.column_config.NumberColumn("Gols Mandante", format="%d",
alignment="center"),

```

```

"rod_gols_visitante": st.column_config.NumberColumn("Gols Visitante", format="%d",
alignment="center"),

"display_visitante": st.column_config.TextColumn("Visitante"),

"clu_link_escudo_visitante": st.column_config.ImageColumn("Escudo"),

},

hide_index=True,

)

return edited, rodada_jogos_df, serie, ano, numero_rodada

def _handle_service_error(e, service_name: str):

 """Helper centralizado para erros de service."""

 if isinstance(e, ValidationError):

 for err in e.errors():

 st.error(err["msg"])

 elif isinstance(e, requests.exceptions.HTTPError):

 status = e.response.status_code if e.response is not None else "?"

 body = e.response.text if e.response is not None else "sem resposta"

 st.error(f"API {service_name} retornou {status}: {body}")

 else:

 st.error(f"Ocorreu um erro inesperado em {service_name}: {str(e)}")

 ...

```

## Página 1 — form\_manut\_rodada\_criar\_rodada.py

```

```python

import streamlit as st

from guards.auth_guard import auth_guard

```

```

from _services.rodada_service import RodadaService

from components.rodada_component import render_rodada_view

auth_guard()

_rodadaService = RodadaService()

edited, df, serie, ano, rodada = render_rodada_view(

    titulo_pagina="Criação de Rodada",

    page_icon="",

    caption="Cadastrar os 10 jogos da rodada, com data, hora, estádio e clubes mandante e
    visitante.",

    fetch_jogos_fn=lambda serie, ano, rodada: _rodadaService.list(

        serie=serie, ano=ano, rodada=rodada, carrega_jogos=False

    ),

)

```

Lógica específica de criação (botão salvar, validações, etc.)

...

...

Página 2 — placar_atualizar.py

```

```python

import streamlit as st

from guards.auth_guard import auth_guard

from _services.rodada_service import RodadaService

```

```

from components.rodada_component import render_rodada_view

auth_guard()

_rodadaService = RodadaService()

edited, df, serie, ano, rodada = render_rodada_view(

 titulo_pagina="Atualizar Placar",

 page_icon="",

 caption="Atualize os placares dos jogos da rodada.",

 fetch_jogos_fn=lambda serie, ano, rodada: _rodadaService.list_preencher_placar(

 serie=serie, ano=ano, rodada=rodada, carrega_jogos=False

),

)

```

## Lógica específica de atualização de placar (botão atualizar, etc.)

...

...

## Resumo do que mudou

Antes	Depois
~200 linhas duplicadas em cada página	Função render_rodada_view centralizada
Mudança de lógica exige editar 2 arquivos	Mudar só em rodada_component.py
Diferença entre telas = 1 linha de service	Diferença = 1 parâmetro fetch_jogos_fn

O padrão usado é **injeção de dependência via callback**: a função não sabe *qual* service chamar — ela só sabe *que* deve chamar algo que retorna jogos. Cada página injeta o service correto no momento da chamada. Isso mantém o **frontend focado em apresentação** e o **service como camada de acesso à API**, alinhado com a separação de responsabilidades.

## Usuário:

O novo arquivo criado `rodada_component.py` esta com erro na linha:

```
response = fetch_jogos_fn(serie=serie, ano=ano, rodada=numero_rodada)
```

(parameter) `fetch_jogos_fn`: Unknown | None

`fetch_jogos_fn`: função que busca os jogos da rodada.

Como resolver?

## IA:

O problema é que o parâmetro está tipado como `None` (pelo valor `default = None`) e o type checker não consegue inferir que é uma função callable. A solução é anotar o tipo com `Callable` e usar `Optional` para aceitar `None`.

No topo do arquivo `rodada_component.py`, adicione o import:

```
```python
from typing import Callable, List, Optional, Any
```
```

Depois ajuste a assinatura da função. Antes estava:

```
```python
def render_rodada_view(
    *,
    titulo_pagina: str,
    page_icon: str,
```

```
caption: str,  
  
fetch_jogos_fn=None,  
  
):  
...
```

Altere para:

```
```python  

def render_rodada_view(

*,

titulo_pagina: str,

page_icon: str,

caption: str,

fetch_jogos_fn: Callable[..., List[Any]] | None = None,

):
...
```

E logo no início do corpo da função, antes de usar `fetch_jogos_fn`, adicione um guard:

```
```python  
  
if fetch_jogos_fn is None:  
  
    st.error("Nenhuma função de busca de jogos foi fornecida.")  
  
    st.stop()  
  
...
```

Por que isso resolve

Problema	Causa	Solução		
----------	-------	---------	--	--

<code>fetch_jogos_fn:</code> Unknown \	None`	Sem anotação de tipo, o checker infere None pelo default	Anotar com <code>Callable[..., List[Any]]` \</code>	None`
Risco de chamar <code>None(...)</code>	Se alguém esquecer de passar o parâmetro	Guard com is None antes de usar		

Se você estiver usando **Python 3.9 ou anterior**, a sintaxe | não funciona. Nesse caso use Optional :

```
```python
from typing import Callable, List, Optional, Any

def render_rodada_view(
 *,
 titulo_pagina: str,
 page_icon: str,
 caption: str,
 fetch_jogos_fn: Optional[Callable[..., List[Any]]] = None,
):
 ...
```

O `Callable[..., List[Any]]` diz ao type checker: *"aceita qualquer função, independente dos parâmetros, que retorne uma lista"*. Os ... indicam que a assinatura dos argumentos não é verificada estaticamente — o que é adequado aqui, já que cada página passa um lambda com serie, ano e rodada.

## Usuário:

No projeto Brasileiro frontend com Streamlit, carreguei o dataframe:

```
edited = st.dataframe(
```



```
rodada_jogos_df,

key="RodadaJogos",

on_select="ignore",

height="auto",

selection_mode=["single-row"],

column_order=[

"rod_rodada",

"rod_data",

"est_nome",

"display_mandante",

"clu_link_escudo_mandante",

"rod_gols_mandante",

"rod_gols_visitante",

"clu_link_escudo_visitante",

"display_visitante",

],

column_config={

"rod_rodada": st.column_config.NumberColumn("Rodada", format="%d", alignment="center"),

"rod_data": st.column_config.DateColumn("Data", format="DD/MM/YYYY HH:mm",

alignment="center"),

"rod_sequencia": st.column_config.NumberColumn("Sequência", format="%d"),

"est_nome": st.column_config.TextColumn("Estádio"),

"display_mandante": st.column_config.TextColumn("Mandante"),

"clu_link_escudo_mandante": st.column_config.ImageColumn("Escudo"),

"rod_gols_mandante": st.column_config.NumberColumn("Gols Mandante", format="%d",

alignment="center"),
```

```

"rod_gols_visitante": st.column_config.NumberColumn("Gols Visitante", format="%d",
alignment="center"),

"display_visitante": st.column_config.TextColumn("Visitante"),

"clu_link_escudo_visitante": st.column_config.ImageColumn("Escudo"),

"rod_pontos_mandante": st.column_config.NumberColumn("Pontos Mandante", format="%d"),

"rod_pontos_visitante": st.column_config.NumberColumn("Pontos Visitante", format="%d"),

"rod_partida_finalizada": st.column_config.TextColumn("PartidaFinalizada"),

"rod_calculou_classificacao": st.column_config.TextColumn("Classificação Calculada"),

},

hide_index=True

)

```

Através do st.dialog quero fazer um loop no dataframe:

- Cada linha quero mostrar e não digitar: rodada, data, clube mandante e clube visitante
- Primeiro loop quero mostra todos os jogos para atualizar o placar
- segundo loop quero digitar paa cada linha os gols do clube mandante e os glos do clube visitante

## IA:

A solução é usar @st.dialog com dois loops: o primeiro apenas exhibe os dados (somente leitura) e o segundo cria os st.number\_input para os gols.

## Código do diálogo

```

```python

@st.dialog("Atualizar Placares da Rodada")

def dialog_atualizar_placar(df: pd.DataFrame):

if df.empty:

st.warning("Nenhum jogo encontrado para esta rodada.")

```

return

=== 1º LOOP: mostrar todos os jogos (somente leitura) ===

```
st.markdown("##### Jogos da Rodada")
```

```
for idx, row in df.iterrows():
```

```
    rodada = int(row["rod_rodada"]) if pd.notna(row["rod_rodada"]) else ""
```

```
    data = row["rod_data"].strftime("%d/%m/%Y %H:%M") if pd.notna(row["rod_data"]) else "—"
```

```
    mandante = row.get("display_mandante", "—")
```

```
    visitante = row.get("display_visitante", "—")
```

```
    cols = st.columns([1, 2, 3, 0.5, 3])
```

```
    with cols[0]:
```

```
        st.caption(f"{rodada}")
```

```
    with cols[1]:
```

```
        st.caption(data)
```

```
    with cols[2]:
```

```
        st.caption(mandante)
```

```
    with cols[3]:
```

```
        st.caption("")
```

```
    with cols[4]:
```

```
        st.caption(visitante)
```

```
    st.divider()
```

=== 2º LOOP: inputs para gols de cada jogo ===

```
st.markdown("##### Informar Placares")
```

```
placares = []
```

```

for idx, row in df.iterrows():

    mandante = row.get("display_mandante", "")

    visitante = row.get("display_visitante", "")

    gols_m_atual = int(row["rod_gols_mandante"]) if pd.notna(row.get("rod_gols_mandante")) else 0

    gols_v_atual = int(row["rod_gols_visitante"]) if pd.notna(row.get("rod_gols_visitante")) else 0

    cols = st.columns([3, 1, 0.3, 1, 3])

    with cols[0]:

        st.text(mandante)

    with cols[1]:

        gols_m = st.number_input(

            "Gols Mandante",

            min_value=0,

            value=gols_m_atual,

            key=f"gols_mand_{idx}",

            label_visibility="collapsed",

        )

    with cols[2]:

        st.markdown("##### x")

    with cols[3]:

        gols_v = st.number_input(

            "Gols Visitante",

            min_value=0,

            value=gols_v_atual,

            key=f"gols_vis_{idx}",

            label_visibility="collapsed",

```

```

)

with cols[4]:

    st.text(visitante)

    placares.append({

        "idx": idx,

        "gols_mandante": gols_m,

        "gols_visitante": gols_v,

    })

if st.button("Salvar Placares", type="primary", use_container_width=True):

    for p in placares:

        df.loc[p["idx"], "rod_gols_mandante"] = p["gols_mandante"]

        df.loc[p["idx"], "rod_gols_visitante"] = p["gols_visitante"]

    st.session_state["placares_atualizados"] = df

    st.session_state["toast"] = {

        "message": "Placares atualizados com sucesso!",

        "icon": "",

    }

    st.rerun()

...

```

Como chamar o diálogo na página principal

Logo abaixo do `st.dataframe(...)`, adicione o botão que abre o diálogo:

```

```python

edited = st.dataframe(

 rodada_jogos_df,

```

**... seus parâmetros existentes ...**

)

## **Botão para abrir o dialog de atualização de placares**

```
if st.button(" Atualizar Placares", type="primary"):
```

```
 dialog_atualizar_placar(rodada_jogos_df)
```

## **Recuperar placares salvos pelo dialog**

```
if "placares_atualizados" in st.session_state:
```

```
 rodada_jogos_df = st.session_state.pop("placares_atualizados")
```

## **Aqui você pode chamar o service para persistir na API**

```
_rodadaService.atualizar_placar(rodada_jogos_df)
```

```
...
```

## **Como funciona o fluxo**

<b>Etapa</b>	<b>O que acontece</b>
1º loop	Percorre cada linha do DataFrame e exibe rodada, data, mandante e visitante como st.caption (somente leitura)
2º loop	Percorre novamente cada linha e cria dois st.number_input por jogo — um para gols mandante, outro para gols visitante

Pré-preenchimento	Se rod_gols_mandante / rod_gols_visitante já tiverem valor no DataFrame, o number_input já abre com esse valor (value=gols_m_atual)
Salvar	Ao clicar no botão, o DataFrame é atualizado com os placares e guardado em st.session_state["placares_atualizados"]
Toast	A mensagem de sucesso é colocada em st.session_state["toast"] e exibida após o st.rerun(), no canto superior direito

> **Detalhe importante:** cada st.number\_input tem uma key única (f"gols\_mand\_{idx}" e f"gols\_vis\_{idx}"), onde idx é o índice da linha no DataFrame. Isso garante que o Streamlit mantenha o estado de cada input individualmente durante as reruns do diálogo.

## Usuário:

Faça esta lógica a partir :

```
import streamlit as st

import pandas as pd

from _services.estadio_service import EstadioService

from _services.clube_service import ClubeService

from _services.rodada_service import RodadaService

from models.rodada_dto import UltimaRodadaResponseSchema, CriarRodadaSchema

from pydantic import ValidationError

from guards.auth_guard import auth_guard

import requests

from helpers.errorHelper import message_error

from datetime import datetime, time

from typing import List
```

auth\_guard()

**Este trecho de código tem o objetivo de mostrar, por exemplo, a mensagem na inclusão "message": "Transação incluída com sucesso".**

**porque ao realizar a inclusão com sucesso, colocar a mensagem em session\_state e executar o comando "st.rerun()".**

**Este comando recarrega a página e como a variável "toast" esta carregando, então vai aparecer no canto superior direito a mensagem**

```
toast = st.session_state.pop("toast", None)
```

```
if toast:
```

```
st.toast(toast["message"], icon=toast["icon"])
```

```
st.set_page_config(
```

```
page_title="Criação de Rodada",
```

```
page_icon="",
```

```
layout="wide",
```

```
initial_sidebar_state="auto",
```

```
)
```

```
st.caption("Cadastrar os 10 jogos da rodada, com data, hora, estádio e clubes mandante e visitante.")
```

## 1. Services

```
_estadioService = EstadioService()
```

```
_clubeService = ClubeService()
```



```

_rodadaService = RodadaService()

_rodadaListaService = RodadaService()

_ultimaRodadaService = RodadaService()

param_serie, param_ano, param_rodada, param_jogos_anteriores = st.columns([1, 1, 1, 1])

with param_serie:

serie = st.selectbox("Série", ["A", "B"])

with param_ano:

anos = list(range(datetime.now().year, datetime.now().year + 1))

ano = st.selectbox("Ano", anos, index=anos.index(2026))

```

## 2. Detectar primeira carga ou troca de série

```

first_load = "serie_anterior" not in st.session_state

serie_changed = st.session_state.get("serie_anterior") != serie

rodada_recem_incluida = st.session_state.pop("rodada_incluida", False)

st.session_state["serie_anterior"] = serie

```

## 4. Buscar a última rodada com a série/ano SELECIONADOS

```

ultima_rodada_cadastrada = UltimaRodadaResponseSchema(

ultima_rodada_cadastrada=0,

proxima_rodada=0

)

try:

response = _ultimaRodadaService._list_ultima_rodada(serie=serie,ano=ano)

ultima_rodada_cadastrada = response if response else UltimaRodadaResponseSchema(

ultima_rodada_cadastrada=0,

```

```

proxima_rodada=0

)

except ValidationError as e:

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

status = e.response.status_code if e.response is not None else "?"

body = e.response.text if e.response is not None else "sem resposta"

st.error(f"API UltimaRodadaService retornou {status}: {body}")

except Exception as e:

st.error(f'Ocorreu um erro inesperado: {str(e)}')

```

### 3. Se primeira carga ou troca de série, forçar number\_input para a última rodada

```

if first_load or serie_changed:

rodada_inicial = ultima_rodada_cadastrada.ultima_rodada_cadastrada

if rodada_inicial > 0:

st.session_state["numero_rodada"] = rodada_inicial

else:

st.session_state["numero_rodada"] = 1

```

### 4. Renderizar o number\_input com key (pega o valor de session\_state)

```

with param_rodada:

numero_rodada = st.number_input(

"Rodada",

```

```

min_value=1,

max_value=38,

key="numero_rodada"

)

with param_jogos_anteriores:

carrega_rodada_anteriores = st.selectbox("Carrega jogos anteriores", ["Não", "Sim"])

```

## 5. Buscar jogos da rodada selecionada

```

rows_jogos_da_rodada = []

try:

response = _rodadaListaService.jogos_nao_finalizados_list(serie=serie,ano=ano,
rodada=numero_rodada, carrega_rodada_anteriores=carrega_rodada_anteriores)

rows_jogos_da_rodada = [x.model_dump() for x in response]

except ValidationError as e:

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

status = e.response.status_code if e.response is not None else "?"

if status == 404:

rows_jogos_da_rodada = []

st.error("Rodada informada não esta cadastrada.")

st.stop

else:

body = e.response.text if e.response is not None else "sem resposta"

st.error(f'API rodadaListaService.list retornou {status}: {body}')

except Exception as e:

```

```
st.error(f'Ocorreu um erro inesperado rodadaListaService.list: {str(e)}')
```

## 6. Buscar estádios e clubes (continua igual)

```
rows_clube = []
```

```
rows_estadio = []
```

```
try:
```

```
response = _estadioService.list(nome_estadio="", nome_cidade="", uf="")
```

```
rows_estadio = [x.model_dump() for x in response]
```

```
except ValidationError as e:
```

```
for err in e.errors():
```

```
st.error(err["msg"])
```

```
except requests.exceptions.HTTPError as e:
```

```
status = e.response.status_code if e.response is not None else "?"
```

```
body = e.response.text if e.response is not None else "sem resposta"
```

```
st.error(f'API estadioService.list retornou {status}: {body}')
```

```
except Exception as e:
```

```
st.error(f'Ocorreu um erro inesperado estadioService.list: {str(e)}')
```

```
try:
```

```
response = _clubeService.list(serie=serie, nome="")
```

```
rows_clube = [x.model_dump() for x in response]
```

```
except ValidationError as e:
```

```
for err in e.errors():
```

```
st.error(err["msg"])
```

```
except requests.exceptions.HTTPError as e:
```

```
status = e.response.status_code if e.response is not None else "?"
```

```
body = e.response.text if e.response is not None else "sem resposta"

st.error(f"API clubeService.list retornou {status}: {body}")

except Exception as e:

st.error(f'Ocorreu um erro inesperado clubeService.list: {str(e)}')

estadio_opcoes = {

row["est_nome"]: row["est_id"]

for row in rows_estadio

if row.get("est_nome")

}

clube_opcoes = {

row["clu_nome"]: row["clu_sigla"]

for row in rows_clube

if row.get("clu_nome")

}

if not estadio_opcoes:

st.error("Nenhum estádio foi encontrado.")

st.stop()

if not clube_opcoes:

st.error("Nenhum clube foi encontrado.")

st.stop()

estadio_lista = [""] + list(estadio_opcoes.keys())

clube_lista = [""] + list(clube_opcoes.keys())

rodada_jogos_df = pd.DataFrame(rows_jogos_da_rodada)
```

# Garantir os tipos corretos

```
if not rodada_jogos_df.empty:
```

```
 rodada_jogos_df["rod_rodada"] = pd.to_numeric(
```

```
 rodada_jogos_df["rod_rodada"],
```

```
 errors="coerce"
```

```
)
```

```
 rodada_jogos_df["rod_data"] = pd.to_datetime(
```

```
 rodada_jogos_df["rod_data"],
```

```
 errors="coerce"
```

```
)
```

```
 rodada_jogos_df["rod_gols_mandante"] = pd.to_numeric(
```

```
 rodada_jogos_df["rod_gols_mandante"],
```

```
 errors="coerce"
```

```
)
```

```
 rodada_jogos_df["rod_gols_visitante"] = pd.to_numeric(
```

```
 rodada_jogos_df["rod_gols_visitante"],
```

```
 errors="coerce"
```

```
)
```

```
 rodada_jogos_df["display_mandante"] = (
```

```
 rodada_jogos_df["clube_clu_sigla_mandante"]
```

```
 + " - "
```

```
 + rodada_jogos_df["clu_nome_mandante"]
```

```
)
```

```
 rodada_jogos_df["display_visitante"] = (
```

```
 rodada_jogos_df["clube_clu_sigla_visitante"]
```

```
+ " - "
```

```
+ rodada_jogos_df["clu_nome_visitante"]
```

```
)
```

```
st.write("COLUMNS:",
rodada_jogos_df.columns.tolist())
```

```
st.write("DTYPES:", rodada_jogos_df.dtypes)
```

```
for coluna in rodada_jogos_df.columns:
```

```
st.write(
```

```
f"COLUNA: {coluna}",
```

```
rodada_jogos_df[coluna].tolist()
```

```
)
```

```
edited = st.dataframe(
```

```
rodada_jogos_df,
```

```
key="RodadaJogos",
```

```
on_select="ignore",
```

```
height="auto",
```

```
selection_mode=["single-row"],
```

```
column_order=[
```

```
"rod_rodada",
```

```
"rod_data",
```

```
"est_nome",

"display_mandante",

"clu_link_escudo_mandante",

"rod_gols_mandante",

"rod_gols_visitante",

"clu_link_escudo_visitante",

"display_visitante",

],

column_config={

"rod_rodada": st.column_config.NumberColumn("Rodada", format="%d", alignment="center"),

"rod_data": st.column_config.DateColumn("Data", format="DD/MM/YYYY HH:mm",

alignment="center"),

"rod_sequencia": st.column_config.NumberColumn("Sequência", format="%d"),

"est_nome": st.column_config.TextColumn("Estádio"),

"display_mandante": st.column_config.TextColumn("Mandante"),

"clu_link_escudo_mandante": st.column_config.ImageColumn("Escudo"),

"rod_gols_mandante": st.column_config.NumberColumn("Gols Mandante", format="%d",

alignment="center"),

"rod_gols_visitante": st.column_config.NumberColumn("Gols Visitante", format="%d",

alignment="center"),

"display_visitante": st.column_config.TextColumn("Visitante"),

"clu_link_escudo_visitante": st.column_config.ImageColumn("Escudo"),

"rod_pontos_mandante": st.column_config.NumberColumn("Pontos Mandante", format="%d"),

"rod_pontos_visitante": st.column_config.NumberColumn("Pontos Visitante", format="%d"),

"rod_partida_finalizada": st.column_config.TextColumn("Partida Finalizada"),

"rod_calculou_classificacao": st.column_config.TextColumn("Classificação Calculada"),

},
```



```
hide_index=True
```

```
)
```

```
col1 = st.columns(1, gap="small")[0]
```

```
with col1:
```

```
add_clicked = st.button(" Informar placares dos jogos", type="primary", use_container_width=True)
```

## IA:

Vou implementar o `@st.dialog` com os dois loops integrados ao seu código existente.

```
```python
```

```
@st.dialog("Atualizar Placares da Rodada")
```

```
def dialog_atualizar_placar(df: pd.DataFrame):
```

```
if df.empty:
```

```
st.warning("Nenhum jogo encontrado para esta rodada.")
```

```
return
```

=== 1º LOOP: mostrar todos os jogos (somente leitura) ===

```
st.markdown("##### Jogos da Rodada")
```

```
for idx, row in df.iterrows():
```

```
rodada = int(row["rod_rodada"]) if pd.notna(row.get("rod_rodada")) else ""
```

```
data = (
```

```
row["rod_data"].strftime("%d/%m/%Y %H:%M")
```

```
if pd.notna(row.get("rod_data"))
```

```
else "—"
```

```
)
```

```

mandante = row.get("display_mandante", "—")

visitante = row.get("display_visitante", "—")

cols = st.columns([1, 2.5, 3, 0.3, 3])

with cols[0]:

    st.caption(f"Rodada {rodada}")

with cols[1]:

    st.caption(data)

with cols[2]:

    st.caption(mandante)

with cols[3]:

    st.markdown("##### ")

with cols[4]:

    st.caption(visitante)

st.divider()

```

=== 2º LOOP: inputs para gols de cada jogo ===

```

st.markdown("##### Informar Placares")

placares = []

for idx, row in df.iterrows():

    mandante = row.get("display_mandante", "—")

    visitante = row.get("display_visitante", "—")

    gols_m_atual = (

        int(row["rod_gols_mandante"])

        if pd.notna(row.get("rod_gols_mandante"))

        else 0
    )

```

```

)

gols_v_atual = (

int(row["rod_gols_visitante"])

if pd.notna(row.get("rod_gols_visitante"))

else 0

)

cols = st.columns([3, 1, 0.3, 1, 3])

with cols[0]:

st.text(mandante)

with cols[1]:

gols_m = st.number_input(

"Gols Mandante",

min_value=0,

value=gols_m_atual,

key=f"gols_mand_{idx}",

label_visibility="collapsed",

)

with cols[2]:

st.markdown("##### x")

with cols[3]:

gols_v = st.number_input(

"Gols Visitante",

min_value=0,

value=gols_v_atual,

key=f"gols_vis_{idx}",

```

```

label_visibility="collapsed",

)

with cols[4]:

    st.text(visitante)

    placares.append(

    {

    "idx": idx,

    "gols_mandante": gols_m,

    "gols_visitante": gols_v,

    }

    )

    if st.button("Salvar Placares", type="primary", use_container_width=True):

        for p in placares:

            df.loc[p["idx"], "rod_gols_mandante"] = p["gols_mandante"]

            df.loc[p["idx"], "rod_gols_visitante"] = p["gols_visitante"]

        st.session_state["placares_atualizados"] = df

        st.session_state["toast"] = {

            "message": "Placares atualizados com sucesso!",

            "icon": "",

        }

        st.rerun()

```

=== Botão que abre o dialog ===

```

col1 = st.columns(1, gap="small")[0]

with col1:

```

```

add_clicked = st.button(

" Informar placares dos jogos",

type="primary",

use_container_width=True,

)

if add_clicked:

dialog_atualizar_placar(rodada_jogos_df)

```

=== Recuperar placares salvos pelo dialog ===

```

if "placares_atualizados" in st.session_state:

rodada_jogos_df = st.session_state.pop("placares_atualizados")

```

Aqui você pode chamar o service para persistir na API

```

_rodadaService.atualizar_placar(rodada_jogos_df)
...

```

Fluxo completo de execução

Passo	O que acontece
Usuário clica no botão	<code>add_clicked = True</code> ! chama <code>dialog_atualizar_placar(rodada_jogos_df)</code>
1º loop	Percorre o DataFrame e exibe rodada, data, mandante vs visitante como <code>st.caption</code> (somente leitura)

<code>st.divider()</code>	Linha separadora visual entre os dois blocos
2º loop	Percorre novamente e cria dois <code>st.number_input</code> por jogo com key única (<code>gols_mand_{idx}</code> , <code>gols_vis_{idx}</code>)
Pré-preenchimento	Se <code>rod_gols_mandante</code> já tiver valor no DataFrame, o input já abre com ele (<code>value=gols_m_atual</code>)
Botão Salvar	Atualiza o DataFrame, guarda em <code>st.session_state["placares_atualizados"]</code> , dispara toast e <code>st.rerun()</code>
Pós-rerun	O toast aparece no canto superior direito e o DataFrame atualizado fica disponível para persistir via service

Detalhes importantes

- As **keys** dos `number_input` usam o `idx` (índice do DataFrame), garantindo que cada input mantenha seu estado individualmente entre reruns do diálogo.
- O `label_visibility="collapsed"` esconde o label mas mantém acessibilidade — o label ainda existe para screen readers.
- O `st.session_state["placares_atualizados"]` é **popped** (removido ao ler), então só é processado uma vez após o salvamento. Se você precisar persistir via API, descomente a linha `_rodadaService.atualizar_placar(...)` e ajuste para o método correto do seu service.

Usuário:

No arquivo `placar_atualizar.py` :

```
import streamlit as st
```

```
import pandas as pd
```

```
from _services.estadio_service import EstadioService
```

```
from _services.clube_service import ClubeService

from _services.rodada_service import RodadaService

from models.rodada_dto import UltimaRodadaResponseSchema, CriarRodadaSchema

from pydantic import ValidationError

from guards.auth_guard import auth_guard

import requests

from helpers.errorHelper import message_error

from datetime import datetime, time

from typing import List

auth_guard()
```

Este trecho de código tem o objetivo de mostrar, por exemplo, a mensagem na inclusão "message": "Transação incluída com sucesso".

porque ao realizar a inclusão com sucesso, colocar a mensagem em session_state e executar o comando "st.rerun()".

Este comando recarrega a página e como a variável "toast" esta carregando, então vai aparecer no canto superior direito a mensagem

```
toast = st.session_state.pop("toast", None)

if toast:

    st.toast(toast["message"], icon=toast["icon"])

st.set_page_config(

    page_title="Criação de Rodada",

    page_icon="",
```

```
layout="wide",
```

```
initial_sidebar_state="auto",
```

```
)
```

```
st.caption("Cadastrar os 10 jogos da rodada, com data, hora, estádio e clubes mandante e visitante.")
```

1. Services

```
_estadioService = EstadioService()
```

```
_clubeService = ClubeService()
```

```
_rodadaListaService = RodadaService()
```

```
_ultimaRodadaService = RodadaService()
```

```
param_serie, param_ano, param_rodada, param_jogos_anteriores = st.columns([1, 1, 1, 1])
```

```
with param_serie:
```

```
serie = st.selectbox("Série", ["A", "B"])
```

```
with param_ano:
```

```
anos = list(range(datetime.now().year, datetime.now().year + 1))
```

```
ano = st.selectbox("Ano", anos, index=anos.index(2026))
```

2. Detectar primeira carga ou troca de série

```
first_load = "serie_anterior" not in st.session_state
```

```
serie_changed = st.session_state.get("serie_anterior") != serie
```

```
rodada_recem_incluida = st.session_state.pop("rodada_incluida", False)
```

```
st.session_state["serie_anterior"] = serie
```

4. Buscar a última rodada com a série/ano SELECIONADOS


```

ultima_rodada_cadastrada = UltimaRodadaResponseSchema(

ultima_rodada_cadastrada=0,

proxima_rodada=0

)

try:

response = _ultimaRodadaService._list_ultima_rodada(serie=serie,ano=ano)

ultima_rodada_cadastrada = response if response else UltimaRodadaResponseSchema(

ultima_rodada_cadastrada=0,

proxima_rodada=0

)

except ValidationError as e:

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

status = e.response.status_code if e.response is not None else "?"

body = e.response.text if e.response is not None else "sem resposta"

st.error(f"API UltimaRodadaService retornou {status}: {body}")

except Exception as e:

st.error(f'Ocorreu um erro inesperado: {str(e)}")

```

3. Se primeira carga ou troca de série, forçar number_input para a última rodada

```

if first_load or serie_changed:

rodada_inicial = ultima_rodada_cadastrada.ultima_rodada_cadastrada

if rodada_inicial > 0:

st.session_state["numero_rodada"] = rodada_inicial

```

else:

```
st.session_state["numero_rodada"] = 1
```

4. Renderizar o number_input com key (pega o valor de session_state)

with param_rodada:

```
numero_rodada = st.number_input(
```

```
"Rodada",
```

```
min_value=1,
```

```
max_value=38,
```

```
key="numero_rodada"
```

```
)
```

with param_jogos_anteriores:

```
carrega_rodada_anteriores = st.selectbox("Carrega jogos anteriores", ["Não", "Sim"])
```

5. Buscar jogos da rodada selecionada

```
rows_jogos_da_rodada = []
```

try:

```
response = _rodadaListaService.jogos_nao_finalizados_list(serie=serie, ano=ano,  
rodada=numero_rodada, carrega_rodada_anteriores=carrega_rodada_anteriores)
```

```
rows_jogos_da_rodada = [x.model_dump() for x in response]
```

except ValidationError as e:

for err in e.errors():

```
st.error(err["msg"])
```

except requests.exceptions.HTTPError as e:

```
status = e.response.status_code if e.response is not None else "?"
```

```

if status == 404:

    rows_jogos_da_rodada = []

    st.error("Rodada informada não esta cadastrada.")

    st.stop

else:

    body = e.response.text if e.response is not None else "sem resposta"

    st.error(f"API rodadaListaService.jogos_ao_finalizados_list retornou {status}: {body}")

except Exception as e:

    st.error(f"Ocorreu um erro inesperado rodadaListaService.jogos_ao_finalizados_list: {str(e)}")

rodada_jogos_df = pd.DataFrame(rows_jogos_da_rodada)

```

Garantir os tipos corretos

```

if not rodada_jogos_df.empty:

    rodada_jogos_df["rod_rodada"] = pd.to_numeric(

        rodada_jogos_df["rod_rodada"],

        errors="coerce"

    )

    rodada_jogos_df["rod_data"] = pd.to_datetime(

        rodada_jogos_df["rod_data"],

        errors="coerce"

    )

    rodada_jogos_df["rod_gols_mandante"] = pd.to_numeric(

        rodada_jogos_df["rod_gols_mandante"],

        errors="coerce"

    )

```

```
rodada_jogos_df["rod_gols_visitante"] = pd.to_numeric(
rodada_jogos_df["rod_gols_visitante"],
errors="coerce"
)
```

```
rodada_jogos_df["display_mandante"] = (
rodada_jogos_df["clube_clu_sigla_mandante"]
+ " - "
+ rodada_jogos_df["clu_nome_mandante"]
)
```

```
rodada_jogos_df["display_visitante"] = (
rodada_jogos_df["clube_clu_sigla_visitante"]
+ " - "
+ rodada_jogos_df["clu_nome_visitante"]
)
```

```
st.write("COLUMNS:",
rodada_jogos_df.columns.tolist())
```

```
st.write("DTYPES:", rodada_jogos_df.dtypes)
```

```
for coluna in rodada_jogos_df.columns:
```

```
st.write(
```

```
f"COLUNA: {coluna}",
```

```
rodada_jogos_df[coluna].tolist()
```

)

```
edited = st.dataframe(  
    rodada_jogos_df,  
    key="RodadaJogos",  
    on_select="ignore",  
    height="auto",  
    selection_mode=["single-row"],  
    column_order=[  
        "rod_rodada",  
        "rod_data",  
        "est_nome",  
        "display_mandante",  
        "clu_link_escudo_mandante",  
        "rod_gols_mandante",  
        "rod_gols_visitante",  
        "clu_link_escudo_visitante",  
        "display_visitante",  
    ],  
    column_config={  
        "rod_rodada": st.column_config.NumberColumn("Rodada", format="%d", alignment="center"),  
        "rod_data": st.column_config.DateColumn("Data", format="DD/MM/YYYY HH:mm",  
        alignment="center"),  
        "rod_sequencia": st.column_config.NumberColumn("Sequência", format="%d"),  
        "est_nome": st.column_config.TextColumn("Estádio"),  
        "display_mandante": st.column_config.TextColumn("Mandante"),  
        "clu_link_escudo_mandante": st.column_config.ImageColumn("Escudo"),
```

```

"rod_gols_mandante": st.column_config.NumberColumn("Gols Mandante", format="%d",
alignment="center"),

"rod_gols_visitante": st.column_config.NumberColumn("Gols Visitante", format="%d",
alignment="center"),

"display_visitante": st.column_config.TextColumn("Visitante"),

"clu_link_escudo_visitante": st.column_config.ImageColumn("Escudo"),

"rod_pontos_mandante": st.column_config.NumberColumn("Pontos Mandante", format="%d"),

"rod_pontos_visitante": st.column_config.NumberColumn("Pontos Visitante", format="%d"),

"rod_partida_finalizada": st.column_config.TextColumn("PartidaFinalizada"),

"rod_calculou_classificacao": st.column_config.TextColumn("Classificação Calculada"),

},

hide_index=True

)

col1 = st.columns(1, gap="small")[0]

with col1:

add_clicked = st.button(" Informar placares dos jogos", type="primary", use_container_width=True)

No rodada_service.py :

@st.cache_data(show_spinner=False, ttl=30)

def jogos_nao_finalizados_list(_self, serie: str, ano: int, rodada: int, carrega_rodada_anteriores:
str) -> List[JogoFormPlacarSchema]:

url = f'{_self.base_url}/form-placar-rodada/jogos-nao-finalizados/{serie}/{ano}/{rodada}'

params = {}

if serie:

params["serie"] = serie

if ano:

params["ano"] = ano

```

if rodada:

params["rodada"] = rodada

if carrega_rodada_anteriores:

params["carrega_rodada_anteriores"] = True if carrega_rodada_anteriores == "Sim" else False

r = requests.get(url, headers=get_auth_headers(),

params=params,

timeout=_self.timeout)

r.raise_for_status() # Verifica se a resposta foi bem-sucedida (status code 200-299)

Valida e retorna a lista de cidades usando o TypeAdapter criado no construtor

return _self._jogos_nao_finalizados_list.validate_python(r.json() or [])

Ao executar a tela ocorre o erro:

Ocorreu um erro inesperado rodadaListaService.list: 'RodadaService' object has no attribute 'jogos_nao_finalizados_list'

IA:

O problema é que `@st.cache_data` não funciona como decorator diretamente em métodos de instância` em algumas versões do Streamlit. O decorator envolve o método em um objeto `CachedFunc` que não implementa o protocolo de descriptor do Python (`__get__`), fazendo com que o método não seja encontrado na instância.

Solução: separar o cache do método

No `rodada_service.py`, substitua o método atual por esta estrutura:

```
```python
```

```
class RodadaService:
```

```

def __init__(self):

self.base_url = ... # sua URL

self.timeout = ...

self._jogos_nao_finalizados_list = TypeAdapter(List[JogoFormPlacarSchema])

def jogos_nao_finalizados_list(

self, serie: str, ano: int, rodada: int, carrega_rodada_anteriores: str

) -> List[JogoFormPlacarSchema]:

```

## Método público sem decorator — apenas chama a função cacheada

```

return _jogos_nao_finalizados_list_cached(

self.base_url,

self.timeout,

serie,

ano,

rodada,

carrega_rodada_anteriores,

)

```

## Função cacheada FORA da classe (aqui o @st.cache\_data funciona)

```

@st.cache_data(show_spinner=False, ttl=30)

def _jogos_nao_finalizados_list_cached(

base_url: str,

timeout: int,

serie: str,

```



```

ano: int,

rodada: int,

carrega_rodada_anteriores: str,

) -> List[JogoFormPlacarSchema]:

url = f"{base_url}/form-placar-rodada/jogos-nao-finalizados/{serie}/{ano}/{rodada}"

params = {}

if serie:

 params["serie"] = serie

if ano:

 params["ano"] = ano

if rodada:

 params["rodada"] = rodada

if carrega_rodada_anteriores:

 params["carrega_rodada_anteriores"] = carrega_rodada_anteriores == "Sim"

r = requests.get(

 url,

 headers=get_auth_headers(),

 params=params,

 timeout=timeout,

)

r.raise_for_status()

adapter = TypeAdapter(List[JogoFormPlacarSchema])

return adapter.validate_python(r.json() or [])

...

```

## Por que isso resolve

Problema	Causa	Solução
AttributeError: object has no attribute	@st.cache_data envolve o método em CachedFunc, que não implementa __get__, então o Python não encontra o método na instância	Mover o cache para uma função standalone fora da classe
_self com underscore não resolve	O underscore apenas evita o hash de self, mas o problema é o descriptor, não o cache em si	A função externa recebe base_url e timeout como argumentos simples (strings/ints são hasheáveis)
TypeAdapter no __init__	Não funciona fora da instância	Instanciar localmente na função cacheada ou no escopo do módulo

## Padrão para todos os services

Se você tem o mesmo problema em outros métodos do RodadaService, aplique a mesma estrutura:

```
```python  
  
class RodadaService:  
  
    def __init__(self):  
  
        self.base_url = "..."  
  
        self.timeout = 30
```

Método público (sem decorator)

```
def list(self, serie, ano, rodada, carrega_jogos=False):  
  
    return _list_cached(self.base_url, self.timeout, serie, ano, rodada, carrega_jogos)
```

```

def list_preencher_placar(self, serie, ano, rodada, carrega_jogos=False):

return _list_preencher_placar_cached(self.base_url, self.timeout, serie, ano, rodada,
carrega_jogos)

def jogos_nao_finalizados_list(self, serie, ano, rodada, carrega_rodada_anteriores):

return _jogos_nao_finalizados_list_cached(

self.base_url, self.timeout, serie, ano, rodada, carrega_rodada_anteriores

)

```

Funções cacheadas no escopo do módulo

```

@st.cache_data(show_spinner=False, ttl=30)

def _list_cached(base_url, timeout, serie, ano, rodada, carrega_jogos):

...

@st.cache_data(show_spinner=False, ttl=30)

def _list_preencher_placar_cached(base_url, timeout, serie, ano, rodada, carrega_jogos):

...

@st.cache_data(show_spinner=False, ttl=30)

def _jogos_nao_finalizados_list_cached(base_url, timeout, serie, ano, rodada,
carrega_rodada_anteriores):

...

'''

```

A regra é simples: **o método da classe é apenas um thin wrapper** que repassa os argumentos para a função cacheada no escopo do módulo, onde `@st.cache_data` funciona corretamente.

Usuário:

No frontend do Brasileirao com Streamlit tenho `placar_atualizar.py`

Estou informando uma rodada que possui jogos finalizados, ou seja, não há nada para carregar na tela, mostra a mensagem: Já foram informados os placares dos jogos da rodada 1.

E depois mostra a mensagem abaixo.

```
pg.run()functionstreamlit.commands.execution_control.stop()-> 'NoReturn'
```

Stops execution immediately.

Streamlit will not run any statements after `st.stop()`.

We recommend rendering a message to explain why the script has stopped.

Example

```
>>> import streamlit as st

>>>

>>> name = st.text_input("Name")

>>> if not name:

>>>     st.warning('Please input a name.')

>>>     st.stop()

>>> st.success("Thank you for inputting a name.")
```

Como resolver para não aparecer a mensagem?

IA:

O problema está em `st.stop` sem os parênteses `()`. Sem os parênteses, o Python não executa a função — ele apenas referencia o objeto da função, e o Streamlit tenta exibi-lo na tela (mostrando a docstring/signatura).

Onde está o bug no seu código

Na seção que trata o erro 404:

```
```python

if status == 404:

 rows_jogos_da_rodada = []

 st.error("Rodada informada não esta cadastrada.")

 st.stop # ERRO: faltam os parênteses!

...


```

E provavelmente você adicionou uma verificação para jogos já finalizados com o mesmo problema:

```
```python

if not rows_jogos_da_rodada:

    st.info("Já foram informados os placares dos jogos da rodada 1.")

    st.stop      # MESMO ERRO: faltam os parênteses!

...


```

Correção

Adicione os parênteses em **todas** as chamadas de st.stop:

```
```python


```

## Bloco do 404

```
if status == 404:

 rows_jogos_da_rodada = []

 st.error("Rodada informada não esta cadastrada.")

 st.stop() # Agora executa corretamente


```

## Verificação de jogos já finalizados (adicione antes do DataFrame)

```

if not rows_jogos_da_rodada:

st.info(f"Já foram informados os placares dos jogos da rodada {int(numero_rodada)}.")

st.stop() # Agora executa corretamente

'''

```

## Por que isso acontece

Código	O que o Python faz	Resultado na tela
st.stop	Referencia o objeto da função sem chamá-la	Streamlit tenta "exibir" o objeto !' mostra a docstring
st.stop()	Chama a função !' leva a exceção para silenciosamente, StopExecution	Exceção para silenciosamente, sem mostrar nada

O st.stop() funciona levantando uma exceção interna que o Streamlit captura para interromper o script. Sem os parênteses, nenhuma exceção é levantada e o objeto da função é renderizado como se fosse conteúdo da página.

## Bloco corrigido completo do tratamento 404 + verificação de lista vazia

```

```python

```

5. Buscar jogos da rodada selecionada

```

rows_jogos_da_rodada = []

try:

response = _rodadaListaService.jogos_nao_finalizados_list(

serie=serie, ano=ano, rodada=numero_rodada,

```

```

carrega_rodada_anteriores=carrega_rodada_anteriores

)

rows_jogos_da_rodada = [x.model_dump() for x in response]

except ValidationError as e:

    for err in e.errors():

        st.error(err["msg"])

except requests.exceptions.HTTPError as e:

    status = e.response.status_code if e.response is not None else "?"

    if status == 404:

        rows_jogos_da_rodada = []

        st.error("Rodada informada não está cadastrada.")

        st.stop() # com parênteses

    else:

        body = e.response.text if e.response is not None else "sem resposta"

        st.error(f"API rodadaListaService.jogos_ao_finalizados_list retornou {status}: {body}")

except Exception as e:

    st.error(f"Ocorreu um erro inesperado rodadaListaService.jogos_ao_finalizados_list: {str(e)}")

```

Verificar se há jogos para atualizar

```

if not rows_jogos_da_rodada:

    st.info(f"Já foram informados os placares dos jogos da rodada {int(numero_rodada)}.")

    st.stop() # com parênteses

...

```

A regra é simples: **todo `st.stop` deve ter `()` no final**. Sempre que ver a docstring de uma função do Streamlit aparecendo na tela, o primeiro suspeito é um `st.alguma_coisa` sem parênteses.

Usuário:

Tenho no projeto Python com Streamlit a tela clube.py , segue o trecho de código para a inclusão de um clube:

```
@st.dialog("Incluir Clube", width="small")

def add_dialog():

    with st.form("add_form", clear_on_submit=False):

        opcoesCidade = {

            cidade["cid_nome"]: cidade["cid_id"]

            for cidade in rows_cidade

        }

        clu_sigla = st.text_input(

            "Sigla",

            placeholder="Digite a sigla do clube",

            max_chars=3,

            key="add_clube_sigla"

        ).upper()

        clu_nome = st.text_input(

            "Clube",

            placeholder="Digite o nome do clube",

            max_chars=100,

            key="add_clube_nome"

        )

        if clu_nome:

            if not validar_nome(clu_nome):

                st.toast(

                    "Informe caracteres válidos",
```



```
icon=""

)

clu_serie = st.selectbox(

"Série",

["A", "B"],

key="add_clube_serie"

)

clu_link_escudo = st.text_input(

"Link do Escudo",

placeholder="Digite o link da imagem do escudo",

max_chars=255,

key="add_clube_link_escudo"

)

cidade_nome_selecionada = st.selectbox(

"Cidade",

options=list(opcoesCidade.keys()),

index=None,

placeholder="Selecione a cidade ...",

key="add_clube_cidade"

)

cidade_id = opcoesCidade.get(

cidade_nome_selecionada,

0

)

submitted = st.form_submit_button(

"Salvar",
```

```
type="primary",

use_container_width=True

)

if submitted:

    try:

        clube_create = ClubeSchema(

            clu_sigla=clu_sigla,

            clu_nome=clu_nome,

            clu_serie=clu_serie,

            clu_link_escudo=clu_link_escudo,

            cidade_cid_id=cidade_id

        )

        create_clube(clube_create)

        st.session_state["toast"] = {

            "message": "Clube incluído com sucesso",

            "icon": ""

        }

        st.rerun()

    except ValidationError as e:

        for err in e.errors():

            st.toast(err["msg"], icon="")

        st.stop()

    except requests.exceptions.HTTPError as e:

        msg = message_error(e)

        st.toast(msg, icon="")

        st.stop()
```

```
except Exception as e:
```

```
st.toast(
```

```
f'Ocorreu um erro inesperado: {str(e)}",
```

```
icon=""
```

```
)
```

```
st.stop()
```

Crie o código para alteração do clube com base na inclusão, o trecho de código a seguir é para alteração do estádio, utilize como referência:

```
@st.dialog("Alterar Clube", width="small")
```

```
def edit_dialog():
```

```
if selecionados.empty:
```

```
return
```

```
with st.form("edit_form", clear_on_submit=False):
```

```
opcoesCidade = {
```

```
cidade["cid_nome"]: cidade["cid_id"]
```

```
for cidade in rows_cidade
```

```
}
```

```
estadio_id_value = selecionados.iloc[0].get(
    "est_id",
    selecionados.iloc[0].get("id")
)

if estadio_id_value is None:
    st.error("ID do estádio não encontrado")
    return

estadio_id = int(estadio_id_value)

estadio_nome = st.text_input(
    "Estádio",
    value=selecionados.iloc[0]["est_nome"] or ""
)

# Lista de cidades disponíveis

lista_cidades = list(opcoesCidade.keys())
```

Cidade atualmente cadastrada para o estádio

**cidade_nome_atual = selecionados.iloc[0]
["cidade_nome"]**

Localiza a posição da cidade na lista

if cidade_nome_atual in lista_cidades:

indice_cidade_atual = lista_cidades.index(

cidade_nome_atual

)

else:

indice_cidade_atual = 0

cidade_nome_selecionada = st.selectbox(

"Cidade",

options=lista_cidades,

index=indice_cidade_atual

)

```
cidade_id_atual = opcoesCidade.get(
cidade_nome_selecionada,
0
)
```

```
submitted = st.form_submit_button(
"Salvar",
type="primary",
use_container_width=True
)
```

```
if submitted:
```

```
try:
```

```
estadio_update = EstadioUpdate(
est_id=estadio_id,
est_nome=estadio_nome,
```

cidade_cid_id=cidade_id_atual

)

update_estadio(estadio_update)

st.session_state["toast"] = {

"message": "Estádio alterado com sucesso",

"icon": ""

}

st.rerun()

except ValidationError as e:

for err in e.errors():

st.toast(err["msg"], icon="")

st.stop()

except requests.exceptions.HTTPError as e:

msg = message_error(e)

st.toast(msg, icon="")

st.stop()

except Exception as e:

st.toast(

f"Ocorreu um erro inesperado: {str(e)}",

icon=""

)

st.stop()

IA:

```
```python
```

```
@st.dialog("Alterar Clube", width="small")
```

```
def edit_dialog():
```

```
if selecionados.empty:
```

```
return
```

```
with st.form("edit_form", clear_on_submit=False):
```

```
opcoesCidade = {
```

```
cidade["cid_nome"]: cidade["cid_id"]
```

```
for cidade in rows_cidade
```



```

}

clube_id_value = selecionados.iloc[0].get(

"clu_id",

selecionados.iloc[0].get("id")

)

if clube_id_value is None:

st.error("ID do clube não encontrado")

return

clube_id = int(clube_id_value)

clu_sigla = st.text_input(

"Sigla",

value=selecionados.iloc[0]["clu_sigla"] or "",

max_chars=3,

key="edit_clube_sigla"

).upper()

clu_nome = st.text_input(

"Clube",

value=selecionados.iloc[0]["clu_nome"] or "",

max_chars=100,

key="edit_clube_nome"

)

if clu_nome:

if not validar_nome(clu_nome):

st.toast(

"Informe caracteres válidos",

```

```
icon=""
```

```
)
```

## Série atual

```
serie_atual = selecionados.iloc[0]["clu_serie"]
```

```
serie_opcoes = ["A", "B"]
```

```
if serie_atual in serie_opcoes:
```

```
 indice_serie = serie_opcoes.index(serie_atual)
```

```
else:
```

```
 indice_serie = 0
```

```
clu_serie = st.selectbox(
```

```
 "Série",
```

```
 serie_opcoes,
```

```
 index=indice_serie,
```

```
 key="edit_clube_serie"
```

```
)
```

```
clu_link_escudo = st.text_input(
```

```
 "Link do Escudo",
```

```
 value=selecionados.iloc[0]["clu_link_escudo"] or "",
```

```
 max_chars=255,
```

```
 key="edit_clube_link_escudo"
```

```
)
```

## Lista de cidades disponíveis

```
lista_cidades = list(opcoesCidade.keys())
```

# Cidade atualmente cadastrada para o clube

```
cidade_nome_atual = selecionados.iloc[0].get("cidade_nome", "")
```

## Localiza a posição da cidade na lista

```
if cidade_nome_atual in lista_cidades:
```

```
 indice_cidade_atual = lista_cidades.index(
```

```
 cidade_nome_atual
```

```
)
```

```
else:
```

```
 indice_cidade_atual = 0
```

```
cidade_nome_selecionada = st.selectbox(
```

```
 "Cidade",
```

```
 options=lista_cidades,
```

```
 index=indice_cidade_atual,
```

```
 key="edit_clube_cidade"
```

```
)
```

```
cidade_id = opcoesCidade.get(
```

```
 cidade_nome_selecionada,
```

```
 0
```

```
)
```

```
submitted = st.form_submit_button(
```

```
 "Salvar",
```

```
 type="primary",
```

```
 use_container_width=True
```

```
)
```

```
if submitted:

 try:

 clube_update = ClubeUpdate(

 clu_id=clube_id,

 clu_sigla=clu_sigla,

 clu_nome=clu_nome,

 clu_serie=clu_serie,

 clu_link_escudo=clu_link_escudo,

 cidade_cid_id=cidade_id

)

 update_clube(clube_update)

 st.session_state["toast"] = {

 "message": "Clube alterado com sucesso",

 "icon": ""

 }

 st.rerun()

 except ValidationError as e:

 for err in e.errors():

 st.toast(err["msg"], icon="")

 st.stop()

 except requests.exceptions.HTTPError as e:

 msg = message_error(e)

 st.toast(msg, icon="")

 st.stop()

 except Exception as e:
```

```

st.toast(

f'Ocorreu um erro inesperado: {str(e)}',

icon=""

)

st.stop()

...

```

## Diferenças em relação ao add\_dialog

Aspecto	add_dialog (inclusão)	edit_dialog (alteração)
ID do clube	Não existe (será gerado pela API)	Extraído de selecionados.iloc[0]
Sigla	Vazia (placeholder)	Pré-preenchida com value=selecionados.iloc[0]["clu_sigla"]
Nome	Vazio	Pré-preenchido
Série	Sem índice definido	index calculado a partir do valor atual
Link do escudo	Vazio	Pré-preenchido
Cidade	index=None	index calculado pela posição da cidade atual na lista
Schema	ClubeSchema (create)	ClubeUpdate (com clu_id)
Service	create_clube(...)	update_clube(...)

> **Atenção:** verifique se o nome do campo da cidade no DataFrame é cidade\_nome. Se for diferente (ex: cid\_nome), ajuste a linha cidade\_nome\_atual = selecionados.iloc[0].get("cidade\_nome", "") com o nome correto da coluna.

## Usuário:

Python campo `rod_data` é do tipo `datetime`. Separar nos campos `data` (`dd/mm/yyyy`), `hora` (`HH`) e `minuto` (`MM`)

## IA:

Vou analisar o arquivo anexado para entender o contexto completo do código.

Com base no código do arquivo anexado, o campo `rod_data` é um `datetime` que atualmente é exibido como uma string única (`strftime("%d/%m/%Y %H:%M")`). Para separar em três campos editáveis — **data**, **hora** e **minuto** — no diálogo, veja como fica:

## Código com `rod_data` separado em 3 campos

Substitua o bloco do primeiro loop dentro do `add_dialog` por:

```
```python
@st.dialog("Informar placares dos jogos", width="large")
def add_dialog():
    with st.form("rodada_form"):
        for idx, row in rodada_jogos_df.iterrows():
            rodada = int(row["rod_rodada"]) if pd.notna(row["rod_rodada"]) else ""
            mandante = row.get("display_mandante", "")
            visitante = row.get("display_visitante", "")
```

=== Extrair componentes do `rod_data` (datetime) ===

```
rod_data = row["rod_data"]

if pd.notna(rod_data):
    data_value = rod_data.date()
```

```
hora_value = int(rod_data.hour)

minuto_value = int(rod_data.minute)

else:

data_value = datetime.now().date()

hora_value = 0

minuto_value = 0
```

=== Linha 1: rodada, mandante vs visitante (somente leitura) ===

```
cols_header = st.columns([1, 3, 0.5, 3])

with cols_header[0]:

st.caption(f'{rodada}')

with cols_header[1]:

st.caption(mandante)

with cols_header[2]:

st.caption("")

with cols_header[3]:

st.caption(visitante)
```

=== Linha 2: data, hora, minuto separados ===

```
cols_data = st.columns([2, 1, 1])

with cols_data[0]:

data_input = st.date_input(

    "Data",

    value=data_value,

    format="DD/MM/YYYY",
```

```
key=f"data_{idx}",
label_visibility="collapsed",
)

with cols_data[1]:
    hora_input = st.selectbox(
        "Hora",
        options=list(range(0, 24)),
        index=hora_value,
        format_func=lambda h: f"{h:02d}h",
        key=f"hora_{idx}",
        label_visibility="collapsed",
    )

    with cols_data[2]:
        minuto_input = st.selectbox(
            "Minuto",
            options=[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55],
            index=minuto_input_index(minuto_value),
            format_func=lambda m: f"{m:02d}",
            key=f"minuto_{idx}",
            label_visibility="collapsed",
        )

    st.divider()

submitted = st.form_submit_button("Salvar", type="primary", use_container_width=True)

if submitted:
```


... sua lógica de salvamento ...

pass

```
def minuto_input_index(minuto_value: int) -> int:
```

```
    """Retorna o índice mais próximo na lista de minutos [0, 5, 10, ...]."""
```

```
    opcoes = [0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55]
```

```
    for i, m in enumerate(opcoes):
```

```
        if minuto_value <= m:
```

```
            return i
```

```
    return len(opcoes) - 1
```

```
...
```

Como cada campo fica separado

Campo	Tipo	Origem do valor	Exemplo
Data	st.date_input	rod_data.date()	26/08/2026
Hora	st.selectbox (0–23)	rod_data.hour	16h
Minuto	st.selectbox (0, 5, 10, ... 55)	rod_data.minute	30

Como remontar o datetime ao salvar

No if submitted:, para cada jogo você reconstrói o datetime a partir dos três campos:

```
```python
```

```
if submitted:
```

```
jogos_atualizados = []

for idx, row in rodada_jogos_df.iterrows():
```

## Remontar datetime a partir dos 3 campos separados

```
data_str = st.session_state[f"data_{idx}"]

hora = st.session_state[f"hora_{idx}"]

minuto = st.session_state[f"minuto_{idx}"]

rod_data_completo = datetime.combine(data_str, time(hour=hora, minute=minuto))

jogos_atualizados.append({

 "idx": idx,

 "rod_data": rod_data_completo,

 "rod_gols_mandante": st.session_state.get(f"gols_mand_{idx}", 0),

 "rod_gols_visitante": st.session_state.get(f"gols_vis_{idx}", 0),

})
```

## Atualizar o DataFrame

```
for j in jogos_atualizados:

 rodada_jogos_df.loc[j["idx"], "rod_data"] = j["rod_data"]
```

## Persistir via service...

```
st.cache_data.clear()

st.session_state["toast"] = {

 "message": "Jogos atualizados com sucesso",

 "icon": ""

}
```

```
st.rerun()
```

```
...
```

## Detalhes importantes

- A **hora** usa selectbox de 0 a 23 com `format_func` que mostra 16h em vez de 16 — mais amigável para o usuário.
- O **minuto** usa intervalos de 5 em 5 ([0, 5, 10, 15, ... 55]) que é o padrão comum em jogos de futebol. Se precisar de minuto livre, troque por `st.number_input` com `min_value=0`, `max_value=59`.
- A função `minuto_input_index()` mapeia o minuto original (ex: 32) para o índice mais próximo na lista (ex: índice 7 != 35), garantindo que o selectbox não quebre com valores fora da lista.
- O `datetime.combine(data_str, time(hour=hora, minute=minuto))` remonta o `datetime` completo a partir dos três campos separados — note o `import` de `time` já existe no seu código (`from datetime import datetime, time`).

## Usuário:

Os comandos a seguir estão corretos?

## Separação da data e hora

```
rod_data_hora_value = selecionados.iloc[0].get(
 "rod_data",
 selecionados.iloc[0].get("rod_data")
)

if rod_data_hora_value is None:
 st.error("data não encontrada")

return

rod_data_hora_value = pd.to_datetime(rod_data_hora_value).to_pydatetime()

rod_data_input = rod_data_hora_value.date()
```

```
rod_hora_input = int(rod_data_hora_value.hour)

rod_minu_input = int(rod_data_hora_value.minute)
```

## IA:

O código funciona, mas tem uma **redundância desnecessária** na primeira linha e dois **typos nos nomes de variáveis**.

## Problema 1: .get() redundante

```
```python

rod_data_hora_value = selecionados.iloc[0].get(

"rod_data",          # chave que você quer buscar

selecionados.iloc[0].get("rod_data") # default = mesma coisa!

)

```
```

Você está chamando `get("rod_data")` **duas vezes** — a segunda como valor padrão da primeira. Se a chave existir, retorna o valor (e ignora o default). Se não existir, o default também vai ser `None`, porque é a mesma busca. Ou seja, é o mesmo que:

```
```python

rod_data_hora_value = selecionados.iloc[0].get("rod_data")

```
```

## Problema 2: Typos nos nomes

- `rod_hora_input !'` deveria ser `rod_hora_input`
- `rod_minu_input !'` deveria ser `rod_minuto_input`

Não causa erro de execução, mas dificulta a manutenção.

## Código corrigido

```
```python
```

Separação da data e hora

```
rod_data_hora_value = selecionados.iloc[0].get("rod_data")

if rod_data_hora_value is None:

    st.error("Data não encontrada")

    return

rod_data_hora_value = pd.to_datetime(rod_data_hora_value).to_pydatetime()

rod_data_input = rod_data_hora_value.date()

rod_hora_input = int(rod_data_hora_value.hour)

rod_minuto_input = int(rod_data_hora_value.minute)

```
```

### Observação sobre o pd.to\_datetime

Se o DataFrame já passou por `pd.to_datetime(rodada_jogos_df["rod_data"], errors="coerce")` (como está no seu código), então `rod_data_hora_value` já é um `pd.Timestamp`. Nesse caso, `pd.to_datetime()` na linha extra é redundante, mas não causa erro — apenas converte um Timestamp em outro Timestamp. Se quiser otimizar:

```
```python
```

Se rod_data já foi convertida anteriormente no DataFrame:

```
if isinstance(rod_data_hora_value, pd.Timestamp):

    rod_data_hora_value = rod_data_hora_value.to_pydatetime()

else:
```

```
rod_data_hora_value = pd.to_datetime(rod_data_hora_value).to_pydatetime()
```

```
...
```

Mas a versão simplificada (com `pd.to_datetime` sempre) é perfeita e mais segura — cobre os dois cenários.

Usuário:

No projeto frontend Brasileiro em Python com Streamlit tenho a tela `cartao.py` :

```
import streamlit as st

import pandas as pd

from _services.clube_service import ClubeService
from _services.cartao_service import CartaoService
from models.cartao_dto import CartaoSchema, CartaoClubeOut
from pydantic import ValidationError
from guards.auth_guard import auth_guard

import requests

from helpers.errorHelper import message_error

from datetime import datetime, time

from typing import List

from components.theme import apply_custom_css

st.set_page_config(
    page_title="Cartões Amarelos e Vermelhos",
    page_icon="",
    layout="centered",
    initial_sidebar_state="expanded",
)
```

```
apply_custom_css()
```

```
st.title("Cartões Amarelos e Vermelhos")
```

```
st.caption("Gerencie os cartões amarelos e vermelhos. Adicione, edite ou exclua cartões.")
```

1. Services

```
_clubeService = ClubeService()
```

```
_cartaoService = CartaoService()
```

```
_cartaoListaService = CartaoService()
```

```
param_serie, param_ano = st.columns([1, 1])
```

```
with param_serie:
```

```
serie = st.selectbox("Série", ["A", "B"])
```

```
with param_ano:
```

```
anos = list(range(datetime.now().year, datetime.now().year + 1))
```

```
ano = st.selectbox("Ano", anos, index=anos.index(2026))
```

2. Detectar primeira carga ou troca de série

```
first_load = "serie_anterior" not in st.session_state
```

```
serie_changed = st.session_state.get("serie_anterior") != serie
```

```
st.session_state["serie_anterior"] = serie
```

```
rows_clube = []
```

```
try:
```

```
response = _clubeService.list(serie, "")
```

```
rows_clube = [x.model_dump() for x in response]
```

```
except ValidationError as e:
```

```
for err in e.errors():
```

```

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

    status = e.response.status_code if e.response is not None else "?"

    body = e.response.text if e.response is not None else "sem resposta"

    st.error(f"API clubeService.list retornou {status}: {body}")

except Exception as e:

    st.error(f'Ocorreu um erro inesperado clubeService.list: {str(e)}')

clube_opcoes = {

    row["clu_nome"]: row["clu_sigla"]

    for row in rows_clube

    if row.get("clu_nome")

}

if not clube_opcoes:

    st.error("Nenhum clube foi encontrado.")

st.stop()

clube_lista = [""] + list(clube_opcoes.keys())

rows_cartao = []

try:

    response = _cartaoService.list(serie, ano)

    rows_cartao = [x.model_dump() for x in response]

except ValidationError as e: # Erro da validação da models em usuarioDTO.py

    for err in e.errors():

        st.error(err["msg"])

except requests.exceptions.HTTPError as e: # Se ocorrer erro no login da API

    msg = message_error(e)

```



```
st.error(msg)

except Exception as e:

st.error(f'Ocorreu um erro inesperado _cartaoService.list: {str(e)}")

cartaoDF = pd.DataFrame(

rows_cartao,

columns=[

"car_serie",

"car_ano",

"clube_clu_sigla",

"clube_nome",

"clube_link_escudo",

"car_qtd_amarelo",

"car_qtd_vermelho"

]

)

edited = st.dataframe(

cartaoDF,

key="clubes",

on_select="rerun",

selection_mode=["single-row"],

column_order=[

"car_serie",

"car_ano",

"clube_link_escudo",

"clube_clu_sigla",
```

```
"clube_nome",

"car_qtd_amarelo",

"car_qtd_vermelho"

],

column_config={

"car_serie": st.column_config.TextColumn(

"Série",

disabled=True,

alignment="center"

),

"car_ano": st.column_config.NumberColumn(

"Ano",

disabled=True,

format="%d",

alignment="center"

),

"clube_link_escudo": st.column_config.ImageColumn(

"Escudo"

),

"clube_clu_sigla": st.column_config.TextColumn(

"Sigla",

disabled=True,

alignment="center"

),

"clube_nome": st.column_config.TextColumn(

"Clube",
```

```
disabled=True

),

"car_qtd_amarelo": st.column_config.NumberColumn(

    "Amarelos",

    disabled=True,

    format="%d",

    alignment="center"

),

"car_qtd_vermelho": st.column_config.NumberColumn(

    "Vermelhos",

    disabled=True,

    format="%d",

    alignment="center"

)

},

hide_index=True

)
```

Recupera as linhas selecionadas

```
selection = edited.get("selection", {})

rows_selecionados = selection.get("rows", [])
```

Remove índices que não existem mais no DataFrame

```
rows_validos = [

    i for i in rows_selecionados
```

```

if 0 <= i < len(cartaoDF)

]

selecionados = cartaoDF.iloc[rows_validos]

col1, col2, col3, col4, col5 = st.columns(5, gap="small")

with col1:

delete_clicked = st.button(" Gerar Cartões", use_container_width=True, disabled=not
selecionados.empty)

with col2:

delete_clicked = st.button(" Cartões CBF", use_container_width=True, disabled=not
selecionados.empty)

with col3:

add_clicked = st.button(" Incluir", type="primary", use_container_width=True, disabled=not
selecionados.empty)

with col4:

edit_clicked = st.button(" Alterar", use_container_width=True, disabled=len(selecionados) != 1)

with col5:

delete_clicked = st.button(" Excluir", use_container_width=True, disabled=selecionados.empty)

@st.dialog("Incluir Cartões Amarelos e Vermelhos", width="small")

def add_dialog():

with st.form("add_form", clear_on_submit=False):

if not rows_clube:

st.error("Nenhum clube encontrado.")

return

clube_selecionado = st.selectbox(

"Sigla",

options=clube_lista,

```

```
key="clube_clu_sigla"
```

```
)
```

```
car_qtd_amarelo = st.number_input(
```

```
"Cartões Amarelos",
```

```
min_value=0,
```

```
step=1,
```

```
)
```

```
car_qtd_vermelho = st.number_input(
```

```
"Cartões Vermelhos",
```

```
min_value=0,
```

```
step=1,
```

```
)
```

```
submitted = st.form_submit_button(
```

```
"Salvar",
```

```
type="primary",
```

```
use_container_width=True
```

```
)
```

```
if submitted:
```

```
try:
```

```
cartao_create = CartaoSchema(
```

```
car_serie=serie,
```

```
car_ano=ano,
```

```
clube_clu_sigla=clube_selecionado,
```

```
car_qtd_vermelho=int(car_qtd_amarelo),
```

```
car_qtd_amarelo=int(car_qtd_vermelho)
```

)

create_cartao(cartao_create)

st.session_state["toast"] = {

"message": "Cartão incluído com sucesso",

"icon": ""

}

st.rerun()

except ValidationError as e:

for err in e.errors():

st.toast(err["msg"], icon="")

st.stop()

except requests.exceptions.HTTPError as e:

msg = message_error(e)

st.toast(msg, icon="")

st.stop()

except Exception as e:

st.toast(

f'Ocorreu um erro inesperado: {str(e)}",

icon=""

)

st.stop()

if add_clicked:

add_dialog()

if edit_clicked:

edit_dialog()

if delete_clicked:

del_dialog()

```
def create_cartao(cartao: CartaoSchema):
```

```
    result = _cartaoService.create(cartao)
```

```
    return result
```

def update_cartao(cartao: CartaoSchema):

result = _cartaoService.update(cartao)

return result

def delete_cartao(serie: str, ano: int, sigla: str):

result = _cartaoService.delete(serie, ano, sigla)

return result

cartao_service.py :

```
import requests # Importa a biblioteca requests para fazer requisições HTTP
```

```
from dotenv import load_dotenv, find_dotenv # Importa a função load_dotenv para carregar  
variáveis de ambiente
```

```

import os, json # Importa a biblioteca os para acessar variáveis de ambiente

from pydantic import TypeAdapter # Importa o TypeAdapter do Pydantic para validação de tipos

from models.cartao_dto import CartaoSchema, CartaoClubeOut

from typing import List, Optional # Importa List e Optional do módulo typing para tipagem de listas

import streamlit as st # Importa a biblioteca Streamlit para criar a interface do usuário

from helpers.jsonable import _jsonable # Importa a função _jsonable do módulo helpers.jsonable
para tornar objetos JSON serializáveis

from helpers.auth import get_auth_headers

load_dotenv(find_dotenv()) # Carrega as variáveis de ambiente do arquivo .env

class CartaoService:

    def __init__(self, timeout=15.0):

        self.base_url = os.getenv("BASE_URL_API") # Obtém a URL base da API a partir das variáveis
        de ambiente

        self.timeout = timeout # Define o tempo limite para as requisições HTTP

        self._list_adapter = TypeAdapter(list[CartaoClubeOut])

        @st.cache_data(show_spinner=False, ttl=30)

        def list(_self, serie: str, ano: int) -> List[CartaoClubeOut]:

            url = f"{_self.base_url}/cartao/listar/{serie}/{ano}"

            params = {}

            if serie:

                params["serie"] = serie

            if ano:

                params["ano"] = ano

            r = requests.get(url, headers=get_auth_headers(),

                params=params,

```



```
timeout=_self.timeout)
```

```
r.raise_for_status() # Verifica se a resposta foi bem-sucedida (status code 200-299)
```

Valida e retorna a lista de cartao usando o TypeAdapter criado no construtor

```
return _self._list_adapter.validate_python(r.json() or [])
```

```
def create(self, cartao: CartaoSchema) -> Optional[CartaoSchema] | None:
```

```
url = f"{self.base_url}/cartao/incluir"
```

```
payload = _jsonable(cartao.model_dump())
```

```
r = requests.post(url,
```

```
headers=get_auth_headers(),
```

```
data=json.dumps(payload),
```

```
timeout=self.timeout)
```

```
r.raise_for_status()
```

```
data = r.json()
```

```
st.cache_data.clear() # Limpa o cache para garantir que a lista de cidades seja atualizada na próxima consulta
```

```
return None if not data else CartaoSchema.model_validate(data) # Valida e retorna a cidade criada
```

Na inclusão dos cartões para um clube, após preencher a sigla, cartões amarelos e cartões vermelhos e clicar no botão "Salvar", ocorre o erro:

Ocorreu um erro inesperado: 'CartaoService' object has no attribute 'create'

IA:

É o **mesmo problema do `@st.cache\_data` dentro da classe** que vimos antes com RodadaService. O decorator na linha do list quebra a resolução de métodos da instância — e acaba afetando também o create.

Além disso, há um **bug de valores trocados** na criação do schema.

Solução: mesmo padrão — cache fora da classe

No `cartao_service.py`:

```
```python

import requests

from dotenv import load_dotenv, find_dotenv

import os, json

from pydantic import TypeAdapter

from models.cartao_dto import CartaoSchema, CartaoClubeOut

from typing import List, Optional

import streamlit as st

from helpers.jsonable import _jsonable

from helpers.auth import get_auth_headers

load_dotenv(find_dotenv())

class CartaoService:

 def __init__(self, timeout=15.0):

 self.base_url = os.getenv("BASE_URL_API")

 self.timeout = timeout

 self._list_adapter = TypeAdapter(list[CartaoClubeOut])
```

## Método público sem decorator — apenas repassa para a função cacheada

```
def list(self, serie: str, ano: int) -> List[CartaoClubeOut]:
```

```

return _list_cached(self.base_url, self.timeout, serie, ano)

def create(self, cartao: CartaoSchema) -> Optional[CartaoSchema] | None:

url = f"{self.base_url}/cartao/incluir"

payload = _jsonable(cartao.model_dump())

r = requests.post(

url,

headers=get_auth_headers(),

data=json.dumps(payload),

timeout=self.timeout,

)

r.raise_for_status()

data = r.json()

st.cache_data.clear()

return None if not data else CartaoSchema.model_validate(data)

```

## Função cacheada FORA da classe

```

@st.cache_data(show_spinner=False, ttl=30)

def _list_cached(base_url: str, timeout: float, serie: str, ano: int) -> List[CartaoClubeOut]:

url = f"{base_url}/cartao/listar/{serie}/{ano}"

params = {}

if serie:

params["serie"] = serie

if ano:

params["ano"] = ano

r = requests.get(

```

```

url,

headers=get_auth_headers(),

params=params,

timeout=timeout,

)

r.raise_for_status()

adapter = TypeAdapter(list[CartaoClubeOut])

return adapter.validate_python(r.json() or [])

'''

```

## Bug adicional: valores trocados no add\_dialog

No cartao.py, dentro do add\_dialog, os campos estão **invertidos**:

```
```python
```

ERRADO — amarelo e vermelho trocados

```

cartao_create = CartaoSchema(

car_serie=serie,

car_ano=ano,

clube_clu_sigla=clube_selecionado,

car_qtd_vermelho=int(car_qtd_amarelo), # !• recebe amarelo

car_qtd_amarelo=int(car_qtd_vermelho) # !• recebe vermelho

)

'''

```

Corrija para:

```
```python
```

## CORRETO

```
cartao_create = CartaoSchema(

car_serie=serie,

car_ano=ano,

clube_clu_sigla=clube_selecionado,

car_qtd_amarelo=int(car_qtd_amarelo),

car_qtd_vermelho=int(car_qtd_vermelho)

)
```
```

Resumo dos dois problemas

| Problema | Causa | Solução |
|---|--|--|
| 'CartaoService' object has no attribute 'create' | @st.cache_data no método list quebra o descriptor protocol da classe | Mover o cache para função standalone _list_cached fora da classe |
| Cartões amarelos salvos como vermelhos e vice-versa | Variáveis trocadas na construção do CartaoSchema | Atribuir cada variável ao campo correto |

A regra permanece a mesma: **`@st.cache\_data` nunca deve ser aplicado diretamente em métodos de classe.** Sempre extrair para uma função no escopo do módulo.

Usuário:

Na tela construída com Streamlit cartoa.py

```
import streamlit as st
```

```
import pandas as pd

from _services.clube_service import ClubeService

from _services.cartao_service import CartaoService

from models.cartao_dto import CartaoSchema, CartaoClubeOut

from pydantic import ValidationError

from guards.auth_guard import auth_guard

import requests

from helpers.errorHelper import message_error

from datetime import datetime, time

from typing import List

from components.theme import apply_custom_css

st.set_page_config(

    page_title="Cartões Amarelos e Vermelhos",

    page_icon="",

    layout="centered",

    initial_sidebar_state="expanded",

)

apply_custom_css()

st.title("Cartões Amarelos e Vermelhos")

st.caption("Gerencie os cartões amarelos e vermelhos. Adicione, edite ou exclua cartões.")
```

1. Services

```
_clubeService = ClubeService()

_cartaoService = CartaoService()

_cartaoListaService = CartaoService()
```

```

param_serie, param_ano = st.columns([1, 1])

with param_serie:

serie = st.selectbox("Série", ["A", "B"])

with param_ano:

anos = list(range(datetime.now().year, datetime.now().year + 1))

ano = st.selectbox("Ano", anos, index=anos.index(2026))

```

2. Detectar primeira carga ou troca de série

```

first_load = "serie_anterior" not in st.session_state

serie_changed = st.session_state.get("serie_anterior") != serie

st.session_state["serie_anterior"] = serie

rows_clube = []

try:

response = _clubeService.list(serie, "")

rows_clube = [x.model_dump() for x in response]

except ValidationError as e:

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

status = e.response.status_code if e.response is not None else "?"

body = e.response.text if e.response is not None else "sem resposta"

st.error(f"API clubeService.list retornou {status}: {body}")

except Exception as e:

st.error(f'Ocorreu um erro inesperado clubeService.list: {str(e)}')

clube_opcoes = {

```

```

row["clu_nome"]: row["clu_sigla"]

for row in rows_clube

if row.get("clu_nome")

}

if not clube_opcoes:

st.error("Nenhum clube foi encontrado.")

st.stop()

clube_lista = [""] + list(clube_opcoes.keys())

rows_cartao = []

try:

response = _cartaoService.list(serie, ano)

rows_cartao = [x.model_dump() for x in response]

except ValidationError as e: # Erro da validação da models em usuarioDTO.py

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e: # Se ocorrer erro no login da API

msg = message_error(e)

st.error(msg)

except Exception as e:

st.error(f'Ocorreu um erro inesperado _cartaoService.list: {str(e)}')

cartaoDF = pd.DataFrame(

rows_cartao,

columns=[

"car_serie",

"car_ano",

```



```
"clube_clu_sigla",  
"clube_nome",  
"clube_link_escudo",  
"car_qtd_amarelo",  
"car_qtd_vermelho"  
]  
)
```

```
edited = st.dataframe(  
    cartaoDF,  
    key="clubes",  
    on_select="rerun",  
    selection_mode=["single-row"],  
    column_order=[  
        "car_serie",  
        "car_ano",  
        "clube_link_escudo",  
        "clube_clu_sigla",  
        "clube_nome",  
        "car_qtd_amarelo",  
        "car_qtd_vermelho"  
    ],  
    column_config={  
        "car_serie": st.column_config.TextColumn(  
            "Série",  
            disabled=True,
```

```
alignment="center"

),

"car_ano": st.column_config.NumberColumn(

    "Ano",

    disabled=True,

    format="%d",

    alignment="center"

),

"clube_link_escudo": st.column_config.ImageColumn(

    "Escudo"

),

"clube_clu_sigla": st.column_config.TextColumn(

    "Sigla",

    disabled=True,

    alignment="center"

),

"clube_nome": st.column_config.TextColumn(

    "Clube",

    disabled=True

),

"car_qtd_amarelo": st.column_config.NumberColumn(

    "Amarelos",

    disabled=True,

    format="%d",

    alignment="center"

),
```

```
"car_qtd_vermelho": st.column_config.NumberColumn(
    "Vermelhos",
    disabled=True,
    format="%d",
    alignment="center"
)
},
hide_index=True
)
```

Recupera as linhas selecionadas

```
selection = edited.get("selection", {})
rows_selecionados = selection.get("rows", [])
```

Remove índices que não existem mais no DataFrame

```
rows_validos = [
    i for i in rows_selecionados
    if 0 <= i < len(cartaoDF)
]

selecionados = cartaoDF.iloc[rows_validos]

col1, col2, col3, col4, col5 = st.columns(5, gap="small")

with col1:

    delete_clicked = st.button(" Gerar Cartões", use_container_width=True, disabled=not
selecionados.empty)

with col2:
```

```
delete_clicked = st.button(" Cartões CBF", use_container_width=True, disabled=not  
selecionados.empty)
```

```
with col3:
```

```
add_clicked = st.button(" Incluir", type="primary", use_container_width=True, disabled=not  
selecionados.empty)
```

```
with col4:
```

```
edit_clicked = st.button(" Alterar", use_container_width=True, disabled=len(selecionados) != 1)
```

```
with col5:
```

```
delete_clicked = st.button(" Excluir", use_container_width=True, disabled=selecionados.empty)
```

```
@st.dialog("Incluir Cartões Amarelos e Vermelhos", width="small")
```

```
def add_dialog():
```

```
with st.form("add_form", clear_on_submit=False):
```

```
if not rows_clube:
```

```
st.error("Nenhum clube encontrado.")
```

```
return
```

```
clube_selecionado = st.selectbox(
```

```
"Sigla",
```

```
options=clube_lista,
```

```
key="clube_clu_sigla",
```

```
placeholder="Selecione o clube ..."
```

```
)
```

```
car_qtd_amarelo = st.number_input(
```

```
"Cartões Amarelos",
```

```
min_value=0,
```

```
step=1,
```

```
)

car_qtd_vermelho = st.number_input(

"Cartões Vermelhos",

min_value=0,

step=1,

)

submitted = st.form_submit_button(

"Salvar",

type="primary",

use_container_width=True

)

if submitted:

    try:

        cartao_create = CartaoSchema(

            car_serie=serie,

            car_ano=ano,

            clube_clu_sigla=clube_selecionado,

            car_qtd_amarelo=int(car_qtd_amarelo),

            car_qtd_vermelho=int(car_qtd_vermelho)

        )

        create_cartao(cartao_create)

        st.session_state["toast"] = {

            "message": "Cartão incluído com sucesso",

            "icon": ""

        }
```

```
st.rerun()

except ValidationError as e:

    for err in e.errors():

        st.toast(err["msg"], icon="")

    st.stop()

except requests.exceptions.HTTPError as e:

    msg = message_error(e)

    st.toast(msg, icon="")

    st.stop()

except Exception as e:

    st.toast(

        f'Ocorreu um erro inesperado: {str(e)}',

        icon=""

    )

    st.stop()

if add_clicked:

    add_dialog()
```

if edit_clicked:

edit_dialog()

if delete_clicked:

del_dialog()

```
def create_cartao(cartao: CartaoSchema):  
  
    result = _cartaoService.create(cartao)  
  
    return result
```

```
def update_cartao(cartao: CartaoSchema):
```

```
    result = _cartaoService.update(cartao)
```

```
    return result
```

```
def delete_cartao(serie: str, ano: int, sigla: str):
```

```
    result = _cartaoService.delete(serie, ano, sigla)
```

```
    return result
```

A seleção do clube na inclusão através do combobox aparece o nome do clube, mas ao clicar no botão "Salvar", ocorre o erro:

Erro: Sigla inválida: SÃO PAULO. Deve conter exatamente 3 letras maiúsculas. (404)

Como passar para cartao_create a sigla do clube?

IA:

O problema é que o selectbox está retornando o **nome do clube** (porque clube_lista é uma lista de nomes), mas o CartaoSchema espera a **sigla**. Você já tem o dicionário clube_opcoes que mapeia nome !' sigla, basta usá-lo.

Correção no add_dialog

Substitua a construção do cartao_create por:

```
```python
```

```
if submitted:
```

```
try:
```

## Busca a sigla a partir do nome selecionado

```
sigla_clube = clube_opcoes.get(clube_selecionado, "")
```

```
if not sigla_clube:
```

```
st.toast("Selecione um clube válido", icon="")
```

```
st.stop()
```

```
cartao_create = CartaoSchema(
```

```
car_serie=serie,
```

```
car_ano=ano,
```

```
clube_clu_sigla=sigla_clube,
```

```
car_qtd_amarelo=int(car_qtd_amarelo),
```

```
car_qtd_vermelho=int(car_qtd_vermelho)
```

```
)
```

```
create_cartao(cartao_create)
```

## ... resto do código

```
```
```

Por que funciona

| Antes | Depois |
|--|--|
| clube_selecionado = "SÃO PAULO" (nome) | clube_selecionado = "SÃO PAULO" (nome) |
| clube_clu_sigla="SÃO PAULO" !' e sigla clube = clube_opcoes["SÃO PAULO"] | clube_clu_sigla="SAO" !' e sigla clube = clube_opcoes["SAO"] |
| clube_clu_sigla="SAO" !' validação passa | |

O dicionário clube_opcoes já existe no seu código e mapeia exatamente o que você precisa:

```

python

clube_opcoes = {

"SÃO PAULO": "SAO",

"CORINTHIANS": "COR",

"PALMEIRAS": "PAL",

...

}

...

```

O selectbox exibe os nomes (chaves) para o usuário, e no momento de salvar você faz a conversão nome !' sigla com .get().

Usuário:

Realizei as correções de cartoa.py

```

import streamlit as st

import pandas as pd

from _services.clube_service import ClubeService

from _services.cartao_service import CartaoService

from models.cartao_dto import CartaoSchema, CartaoClubeOut

```

```
from pydantic import ValidationError

from guards.auth_guard import auth_guard

import requests

from helpers.errorHelper import message_error

from datetime import datetime, time

from typing import List

from components.theme import apply_custom_css

st.set_page_config(

    page_title="Cartões Amarelos e Vermelhos",

    page_icon="",

    layout="centered",

    initial_sidebar_state="expanded",

)

apply_custom_css()

st.title("Cartões Amarelos e Vermelhos")

st.caption("Gerencie os cartões amarelos e vermelhos. Adicione, edite ou exclua cartões.")
```

1. Services

```
_clubeService = ClubeService()

_cartaoService = CartaoService()

_cartaoListaService = CartaoService()

param_serie, param_ano = st.columns([1, 1])

with param_serie:

    serie = st.selectbox("Série", ["A", "B"])

with param_ano:
```

```
anos = list(range(datetime.now().year, datetime.now().year + 1))
```

```
ano = st.selectbox("Ano", anos, index=anos.index(2026))
```

2. Detectar primeira carga ou troca de série

```
first_load = "serie_anterior" not in st.session_state
```

```
serie_changed = st.session_state.get("serie_anterior") != serie
```

```
st.session_state["serie_anterior"] = serie
```

```
rows_clube = []
```

```
try:
```

```
response = _clubeService.list(serie, "")
```

```
rows_clube = [x.model_dump() for x in response]
```

```
except ValidationError as e:
```

```
for err in e.errors():
```

```
st.error(err["msg"])
```

```
except requests.exceptions.HTTPError as e:
```

```
status = e.response.status_code if e.response is not None else "?"
```

```
body = e.response.text if e.response is not None else "sem resposta"
```

```
st.error(f"API clubeService.list retornou {status}: {body}")
```

```
except Exception as e:
```

```
st.error(f"Ocorreu um erro inesperado clubeService.list: {str(e)}")
```

```
clube_opcoes = {
```

```
row["clu_nome"]: row["clu_sigla"]
```

```
for row in rows_clube
```

```
if row.get("clu_nome")
```

```
}
```

```
if not clube_opcoes:

st.error("Nenhum clube foi encontrado.")

st.stop()

clube_lista = [""] + list(clube_opcoes.keys())

rows_cartao = []

try:

response = _cartaoService.list(serie, ano)

rows_cartao = [x.model_dump() for x in response]

except ValidationError as e: # Erro da validação da models em usuarioDTO.py

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e: # Se ocorrer erro no login da API

msg = message_error(e)

st.error(msg)

except Exception as e:

st.error(f'Ocorreu um erro inesperado _cartaoService.list: {str(e)}')

cartaoDF = pd.DataFrame(

rows_cartao,

columns=[

"car_serie",

"car_ano",

"clube_clu_sigla",

"clube_nome",

"clube_link_escudo",

"car_qtd_amarelo",
```

```
"car_qtd_vermelho"

]

)

edited = st.dataframe(

cartaoDF,

key="clubes",

on_select="rerun",

selection_mode=["single-row"],

column_order=[

"car_serie",

"car_ano",

"clube_link_escudo",

"clube_clu_sigla",

"clube_nome",

"car_qtd_amarelo",

"car_qtd_vermelho"

],

column_config={

"car_serie": st.column_config.TextColumn(

"Série",

disabled=True,

alignment="center"

),

"car_ano": st.column_config.NumberColumn(

>Ano",
```

```
disabled=True,  
  
format="%d",  
  
alignment="center"  
  
)  
  
"clube_link_escudo": st.column_config.ImageColumn(  
  
"Escudo"  
  
)  
  
"clube_clu_sigla": st.column_config.TextColumn(  
  
"Sigla",  
  
disabled=True,  
  
alignment="center"  
  
)  
  
"clube_nome": st.column_config.TextColumn(  
  
"Clube",  
  
disabled=True  
  
)  
  
"car_qtd_amarelo": st.column_config.NumberColumn(  
  
"Amarelos",  
  
disabled=True,  
  
format="%d",  
  
alignment="center"  
  
)  
  
"car_qtd_vermelho": st.column_config.NumberColumn(  
  
"Vermelhos",  
  
disabled=True,  
  
format="%d",
```

```
alignment="center"
```

```
)
```

```
},
```

```
hide_index=True
```

```
)
```

Recupera as linhas selecionadas

```
selection = edited.get("selection", {})
```

```
rows_selecionados = selection.get("rows", [])
```

Remove índices que não existem mais no DataFrame

```
rows_validos = [
```

```
    i for i in rows_selecionados
```

```
    if 0 <= i < len(cartaoDF)
```

```
]
```

```
selecionados = cartaoDF.iloc[rows_validos]
```

```
col1, col2, col3, col4, col5 = st.columns(5, gap="small")
```

```
with col1:
```

```
delete_clicked = st.button(" Gerar Cartões", use_container_width=True, disabled=not  
selecionados.empty)
```

```
with col2:
```

```
delete_clicked = st.button(" Cartões CBF", use_container_width=True, disabled=not  
selecionados.empty)
```

```
with col3:
```

```
add_clicked = st.button(" Incluir", type="primary", use_container_width=True, disabled=not  
selecionados.empty)
```

with col4:

```
edit_clicked = st.button(" Alterar", use_container_width=True,disabled=len(selecionados) != 1)
```

with col5:

```
delete_clicked = st.button(" Excluir", use_container_width=True,disabled=selecionados.empty)
```

```
@st.dialog("Incluir Cartões Amarelos e Vermelhos", width="small")
```

```
def add_dialog():
```

```
with st.form("add_form", clear_on_submit=False):
```

```
if not rows_clube:
```

```
st.error("Nenhum clube encontrado.")
```

```
return
```

```
clube_selecionado = st.selectbox(
```

```
"Sigla",
```

```
options=clube_lista,
```

```
key="clube_clu_sigla",
```

```
placeholder="Selecione o clube ..."
```

```
)
```

```
car_qtd_amarelo = st.number_input(
```

```
"Cartões Amarelos",
```

```
min_value=0,
```

```
step=1,
```

```
)
```

```
car_qtd_vermelho = st.number_input(
```

```
"Cartões Vermelhos",
```

```
min_value=0,
```



```
step=1,

)

submitted = st.form_submit_button(

"Salvar",

type="primary",

use_container_width=True

)

if submitted:

try:

sigla_clube = clube_opcoes.get(clube_selecionado, "")

cartao_create = CartaoSchema(

car_serie=serie,

car_ano=ano,

clube_clu_sigla=sigla_clube,

car_qtd_amarelo=int(car_qtd_amarelo),

car_qtd_vermelho=int(car_qtd_vermelho)

)

create_cartao(cartao_create)

st.session_state["toast"] = {

"message": "Cartão incluído com sucesso",

"icon": ""

}

st.rerun()

except ValidationError as e:
```

```

for err in e.errors():

    st.toast(err["msg"], icon="")

    st.stop()

except requests.exceptions.HTTPError as e:

    msg = message_error(e)

    st.toast(msg, icon="")

    st.stop()

except Exception as e:

    st.toast(

        f'Ocorreu um erro inesperado: {str(e)}',

        icon=""

    )

    st.stop()

if add_clicked:

    add_dialog()

```

if edit_clicked:

edit_dialog()

if delete_clicked:

del_dialog()

```

def create_cartao(cartao: CartaoSchema):

    result = _cartaoService.create(cartao)

```

```
return result
```

```
def update_cartao(cartao: CartaoSchema):
```

```
    result = _cartaoService.update(cartao)
```

```
    return result
```

```
def delete_cartao(serie: str, ano: int, sigla: str):
```

```
    result = _cartaoService.delete(serie, ano, sigla)
```

```
    return result
```

Desenvolva o código para alterar e excluir cartão

IA:

Aqui está o código completo para **alteração** e **exclusão** de cartões, seguindo o mesmo padrão do `add_dialog` e da referência de estádio que você usou antes.

edit_dialog — Alterar Cartão

```
```python
```

```
@st.dialog("Alterar Cartões Amarelos e Vermelhos", width="small")
```

```
def edit_dialog():
```

```
 if selecionados.empty:
```

```
 return
```

```
with st.form("edit_form", clear_on_submit=False):
```

## Dados da linha selecionada

```
clube_sigla_atual = selecionados.iloc[0]["clube_clu_sigla"]
```

```
clube_nome_atual = selecionados.iloc[0]["clube_nome"]
```

```
qtd_amarelo_atual = int(selecionados.iloc[0]["car_qtd_amarelo"]) if pd.notna(selecionados.iloc[0]
["car_qtd_amarelo"]) else 0
```

```
qtd_vermelho_atual = int(selecionados.iloc[0]["car_qtd_vermelho"]) if pd.notna(selecionados.iloc[0]
["car_qtd_vermelho"]) else 0
```

## Mostra o clube (somente leitura — não dá para trocar o clube na alteração)

```
st.text_input(
```

```
"Clube",
```

```
value=clube_nome_atual,
```

```
disabled=True
```

```
)
```

```
car_qtd_amarelo = st.number_input(
```

```
"Cartões Amarelos",
```

```
min_value=0,
```

```
step=1,
```

```
value=qtd_amarelo_atual,
```

```
key="edit_car_qtd_amarelo"
```

```
)
```

```
car_qtd_vermelho = st.number_input(
```

```
"Cartões Vermelhos",
```

```
min_value=0,

step=1,

value=qtd_vermelho_atual,

key="edit_car_qtd_vermelho"

)

submitted = st.form_submit_button(

"Salvar",

type="primary",

use_container_width=True

)

if submitted:

try:

cartao_update = CartaoSchema(

car_serie=serie,

car_ano=ano,

clube_clu_sigla=clube_sigla_atual,

car_qtd_amarelo=int(car_qtd_amarelo),

car_qtd_vermelho=int(car_qtd_vermelho)

)

update_cartao(cartao_update)

st.session_state["toast"] = {

"message": "Cartão alterado com sucesso",

"icon": ""

}

st.rerun()
```

```

except ValidationError as e:

 for err in e.errors():

 st.toast(err["msg"], icon="")

 st.stop()

except requests.exceptions.HTTPError as e:

 msg = message_error(e)

 st.toast(msg, icon="")

 st.stop()

except Exception as e:

 st.toast(

 f"Ocorreu um erro inesperado: {str(e)}",

 icon=""

)

 st.stop()

...

```

## del\_dialog — Excluir Cartão

```

```python

@st.dialog("Excluir Cartão", width="small")

def del_dialog():

    if selecionados.empty:

        return

    clube_nome = selecionados.iloc[0]["clube_nome"]

    clube_sigla = selecionados.iloc[0]["clube_clu_sigla"]

```

```
qtd_amarelo = int(selecionados.iloc[0]["car_qtd_amarelo"]) if pd.notna(selecionados.iloc[0]
["car_qtd_amarelo"]) else 0
```

```
qtd_vermelho = int(selecionados.iloc[0]["car_qtd_vermelho"]) if pd.notna(selecionados.iloc[0]
["car_qtd_vermelho"]) else 0
```

```
st.warning(f"Deseja realmente excluir os cartões do clube {clube_nome}?")
```

```
st.markdown(f"""
```

```
• Série: {serie}
```

```
• Ano: {ano}
```

```
• Amarelos: {qtd_amarelo}
```

```
• Vermelhos: {qtd_vermelho}
```

```
""")
```

```
col_confirm, col_cancel = st.columns(2, gap="small")
```

```
with col_confirm:
```

```
confirmed = st.button(
```

```
" Excluir",
```

```
type="primary",
```

```
use_container_width=True
```

```
)
```

```
with col_cancel:
```

```
cancelled = st.button(
```

```
"Cancelar",
```

```
use_container_width=True
```

```
)
```

```
if confirmed:
```

```
try:
```

```
delete_cartao(serie, ano, clube_sigla)
```

```

st.session_state["toast"] = {

    "message": "Cartão excluído com sucesso",

    "icon": ""

}

st.rerun()

except requests.exceptions.HTTPError as e:

    msg = message_error(e)

    st.toast(msg, icon="")

    st.stop()

except Exception as e:

    st.toast(

        f'Ocorreu um erro inesperado: {str(e)}',

        icon=""

    )

    st.stop()

if cancelled:

    st.rerun()

'''

```

Descomentar e ativar as chamadas

No final do arquivo, troque as linhas comentadas por:

```

```python

if add_clicked:

 add_dialog()

```



```
if edit_clicked:
```

```
 edit_dialog()
```

```
if delete_clicked:
```

```
 del_dialog()
```

```
...
```

## Descomentar as funções de service

```
```python
```

```
def create_cartao(cartao: CartaoSchema):
```

```
    result = _cartaoService.create(cartao)
```

```
    return result
```

```
def update_cartao(cartao: CartaoSchema):
```

```
    result = _cartaoService.update(cartao)
```

```
    return result
```

```
def delete_cartao(serie: str, ano: int, sigla: str):
```

```
    result = _cartaoService.delete(serie, ano, sigla)
```

```
    return result
```

```
...
```

Diferenças entre os três diálogos

Aspecto	add_dialog	edit_dialog	del_dialog
---------	------------	-------------	------------

Clube	selectbox editável	text_input desabilitado (só leitura)	Texto de confirmação
Amarelos/Vermelhos	Vazio (value=0)	Pré-preenchido da linha selecionada	Somente exibição
Schema	CartaoSchema (create)	CartaoSchema (update)	Sem schema — chama delete(serie, ano, sigla)
Service	create_cartao(...)	update_cartao(...)	delete_cartao(serie, ano, sigla)
Confirmação	Botão Salvar direto	Botão Salvar direto	Botão Excluir + Cancelar

> **Atenção:** verifique se o CartaoService já tem os métodos update e delete implementados. Se não tiver, adicione seguindo o mesmo padrão do create — sem @st.cache_data diretamente no método da classe.

Usuário:

Implementar os botões "Gerar Cartões" e "Cartões CBF".

Estes dois botões irão chamar os endpoints vinculados na chamada:

```
def gerar_cartões(serie: str, ano: int):
    result = _cartaoService.criar_cartoes(serie, ano)
    return result

def cartoes_cbf(serie: str, ano: int):
    result = _cartaoService.cbf(serie, ano)
    return result
```

IA:

Vou analisar o arquivo anexado para entender o contexto completo do código atual.

Aqui estão os dois botões implementados, com diálogos de confirmação e tratamento de erros. Substitua os botões atuais (col1 até col5) e as funções de service no final do arquivo.

1. Substituir os botões atuais pelos novos

Troque o bloco col1, col2, col3, col4, col5 = st.columns(...) por:

```
```python
```

```
col1, col2, col3, col4, col5 = st.columns(5, gap="small")
```

```
with col1:
```

```
gerar_clicked = st.button(
```

```
" Gerar Cartões",
```

```
use_container_width=True,
```

```
disabled=selecionados.empty
```

```
)
```

```
with col2:
```

```
cbf_clicked = st.button(
```

```
" Cartões CBF",
```

```
use_container_width=True,
```

```
disabled=selecionados.empty
```

```
)
```

```
with col3:
```

```
add_clicked = st.button(
```

```
" Incluir",
```

```
type="primary",
```

```
use_container_width=True,
```

```

disabled=not selecionados.empty

)

with col4:

edit_clicked = st.button(

" Alterar",

use_container_width=True,

disabled=len(selecionados) != 1

)

with col5:

delete_clicked = st.button(

" Excluir",

use_container_width=True,

disabled=selecionados.empty

)

...

```

## 2. Adicionar os diálogos e as chamadas

Logo abaixo do if add\_clicked: add\_dialog(), adicione:

```

```python

if edit_clicked:

edit_dialog()

if delete_clicked:

del_dialog()

```

=== Diálogo: Gerar Cartões ===

```
@st.dialog("Gerar Cartões", width="small")

def gerar_dialog():

    st.warning(

        f"Deseja gerar os cartões para a Série {serie} - {ano} "

        f"do clube {selecionados.iloc[0]['clube_nome']}?"

    )

    col_confirm, col_cancel = st.columns(2, gap="small")

    with col_confirm:

        confirmed = st.button("Gerar", type="primary", use_container_width=True)

    with col_cancel:

        cancelled = st.button("Cancelar", use_container_width=True)

    if confirmed:

        try:

            gerar_cartoes(serie, ano)

            st.cache_data.clear()

            st.session_state["toast"] = {

                "message": "Cartões gerados com sucesso",

                "icon": ""

            }

            st.rerun()

        except requests.exceptions.HTTPError as e:

            msg = message_error(e)

            st.toast(msg, icon="")

    st.stop()
```

```
except Exception as e:
```

```
st.toast(f"Ocorreu um erro inesperado: {str(e)}", icon="")
```

```
st.stop()
```

```
if cancelled:
```

```
st.rerun()
```

=== Diálogo: Cartões CBF ===

```
@st.dialog("Cartões CBF", width="small")
```

```
def cbf_dialog():
```

```
st.warning(
```

```
f"Deseja atualizar os cartões CBF para a Série {serie} - {ano} "
```

```
f"do clube {selecionados.iloc[0]['clube_nome']}?"
```

```
)
```

```
col_confirm, col_cancel = st.columns(2, gap="small")
```

```
with col_confirm:
```

```
confirmed = st.button("Atualizar", type="primary", use_container_width=True)
```

```
with col_cancel:
```

```
cancelled = st.button("Cancelar", use_container_width=True)
```

```
if confirmed:
```

```
try:
```

```
cartoes_cbf(serie, ano)
```

```
st.cache_data.clear()
```

```
st.session_state["toast"] = {
```

```
"message": "Cartões CBF atualizados com sucesso",
```

```
"icon": ""
```

```
}

st.rerun()

except requests.exceptions.HTTPError as e:

    msg = message_error(e)

    st.toast(msg, icon="")

    st.stop()

except Exception as e:

    st.toast(f'Ocorreu um erro inesperado: {str(e)}", icon="")

    st.stop()

if cancelled:

    st.rerun()
```

=== Chamadas dos botões ===

```
if gerar_clicked:

    gerar_dialog()

if cbf_clicked:

    cbf_dialog()
```

=== Funções de service ===

```
def create_cartao(cartao: CartaoSchema):

    result = _cartaoService.create(cartao)

    return result

def update_cartao(cartao: CartaoSchema):

    result = _cartaoService.update(cartao)

    return result
```

```

def delete_cartao(serie: str, ano: int, sigla: str):

result = _cartaoService.delete(serie, ano, sigla)

return result


def gerar_cartoes(serie: str, ano: int):

result = _cartaoService.criar_cartoes(serie, ano)

return result


def cartoes_cbf(serie: str, ano: int):

result = _cartaoService.cbf(serie, ano)

return result

```

Resumo do fluxo de cada botão

Botão	Diálogo	Ação ao confirmar	Toast de sucesso
Gerar Cartões	gerar_dialog	gerar_cartoes(serie, ano) !' _cartaoService.criar_cartoes(...)	"Cartões gerados com sucesso"
Cartões CBF	cbf_dialog	cartoes_cbf(serie, ano) !' _cartaoService.cbf(...)	"Cartões CBF atualizados com sucesso"
Incluir	add_dialog	create_cartao(...)	"Cartão incluído com sucesso"
Alterar	edit_dialog	update_cartao(...)	"Cartão alterado com sucesso"
Excluir	del_dialog	delete_cartao(...)	"Cartão excluído com sucesso"

> **Atenção:** verifique se os métodos `criar_cartoes` e `cbf` já existem no `CartaoService`. Se não existirem, adicione seguindo o mesmo padrão do `create` — **sem** `@st.cache_data` diretamente no método da classe.

Usuário:

No trecho de código a seguir:

```
param_serie, param_ano = st.columns([1, 1])

with param_serie:

    serie = st.selectbox("Série", ["A", "B"])

with param_ano:

    anos = list(range(datetime.now().year, datetime.now().year + 1))

    ano = st.selectbox("Ano", anos, index=anos.index(2026))
```

O selectbox "Ano" somente mostra o ano de 2026, não aparece o ano 2027

IA:

O problema está na linha que gera a lista de anos:

```
```python

anos = list(range(datetime.now().year, datetime.now().year + 1))

```
```

O `range(start, stop)` **exclui o valor** `stop`. Então `range(2026, 2027)` gera apenas `[2026]`. Para incluir 2027, o `stop` precisa ser `+ 2`:

```
```python

anos = list(range(datetime.now().year, datetime.now().year + 2))

```
```

Assim `range(2026, 2028)` gera `[2026, 2027]`.

Código corrigido

```
```python

with param_ano:

 anos = list(range(datetime.now().year, datetime.now().year + 2))

 ano = st.selectbox("Ano", anos, index=anos.index(2026))

...
```
```

Se no futuro quiser mostrar mais anos (ex: 2026, 2027, 2028), basta aumentar o + 2 para + 3 e assim por diante.

Usuário:

Quero habilitar o botão "Gerar Cartões" quando rows_cartao = 0, caso contrário, o botão deve ficar desabilitado.

O botão "Cartões CBF" deve ser habilitado quando rows_cartao != 0; caso contrário, o botão deve ficar desabilitado.

Segue o trecho do código:

```
import streamlit as st

import pandas as pd

from _services.clube_service import ClubeService

from _services.cartao_service import CartaoService

from models.cartao_dto import CartaoSchema, CartaoClubeOut

from pydantic import ValidationError

from guards.auth_guard import auth_guard

import requests

from helpers.errorHelper import message_error

from datetime import datetime, time

from typing import List
```

```
from components.theme import apply_custom_css

st.set_page_config(

    page_title="Cartões Amarelos e Vermelhos",

    page_icon="",

    layout="centered",

    initial_sidebar_state="expanded",

)

apply_custom_css()

st.title("Cartões Amarelos e Vermelhos")

st.caption("Gerencie os cartões amarelos e vermelhos. Adicione, edite ou exclua cartões.")
```

1. Services

```
_clubeService = ClubeService()

_cartaoService = CartaoService()

_cartaoListaService = CartaoService()

param_serie, param_ano = st.columns([1, 1])

with param_serie:

    serie = st.selectbox("Série", ["A", "B"])

with param_ano:

    anos = list(range(datetime.now().year, datetime.now().year + 2))

    ano = st.selectbox("Ano", anos, index=anos.index(2026))
```

2. Detectar primeira carga ou troca de série

```
first_load = "serie_anterior" not in st.session_state

serie_changed = st.session_state.get("serie_anterior") != serie
```

```

st.session_state["serie_anterior"] = serie

rows_clube = []

try:

response = _clubeService.list(serie, "")

rows_clube = [x.model_dump() for x in response]

except ValidationError as e:

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e:

status = e.response.status_code if e.response is not None else "?"

body = e.response.text if e.response is not None else "sem resposta"

st.error(f"API clubeService.list retornou {status}: {body}")

except Exception as e:

st.error(f'Ocorreu um erro inesperado clubeService.list: {str(e)}')

clube_opcoes = {

row["clu_nome"]: row["clu_sigla"]

for row in rows_clube

if row.get("clu_nome")

}

if not clube_opcoes:

st.error("Nenhum clube foi encontrado.")

st.stop()

clube_lista = [""] + list(clube_opcoes.keys())

rows_cartao = []

```

```

try:

response = _cartaoService.list(serie, ano)

rows_cartao = [x.model_dump() for x in response]

except ValidationError as e: # Erro da validação da models em usuarioDTO.py

for err in e.errors():

st.error(err["msg"])

except requests.exceptions.HTTPError as e: # Se ocorrer erro no login da API

msg = message_error(e)

st.error(msg)

except Exception as e:

st.error(f'Ocorreu um erro inesperado _cartaoService.list: {str(e)}')

cartaoDF = pd.DataFrame(

rows_cartao,

columns=[

"car_serie",

"car_ano",

"clube_clu_sigla",

"clube_nome",

"clube_link_escudo",

"car_qtd_amarelo",

"car_qtd_vermelho"

]

)

edited = st.dataframe(

cartaoDF,

```

```
key="clubes",

on_select="rerun",

selection_mode=["single-row"],

column_order=[

"car_serie",

"car_ano",

"clube_link_escudo",

"clube_clu_sigla",

"clube_nome",

"car_qtd_amarelo",

"car_qtd_vermelho"

],

column_config={

"car_serie": st.column_config.TextColumn(

"Série",

disabled=True,

alignment="center"

),

"car_ano": st.column_config.NumberColumn(

>Ano",

disabled=True,

format="%d",

alignment="center"

),

"clube_link_escudo": st.column_config.ImageColumn(

"Escudo"
```

```
),  
  
"clube_clu_sigla": st.column_config.TextColumn(  
  
"Sigla",  
  
disabled=True,  
  
alignment="center"  
  
),  
  
"clube_nome": st.column_config.TextColumn(  
  
"Clube",  
  
disabled=True  
  
),  
  
"car_qtd_amarelo": st.column_config.NumberColumn(  
  
"Amarelos",  
  
disabled=True,  
  
format="%d",  
  
alignment="center"  
  
),  
  
"car_qtd_vermelho": st.column_config.NumberColumn(  
  
"Vermelhos",  
  
disabled=True,  
  
format="%d",  
  
alignment="center"  
  
)  
  
},  
  
hide_index=True  
  
)
```

Recupera as linhas selecionadas

```
selection = edited.get("selection", {})
```

```
rows_selecionados = selection.get("rows", [])
```

Remove índices que não existem mais no DataFrame

```
rows_validos = [
```

```
    i for i in rows_selecionados
```

```
    if 0 <= i < len(cartaoDF)
```

```
]
```

```
selecionados = cartaoDF.iloc[rows_validos]
```

```
col1, col2, col3, col4, col5 = st.columns(5, gap="small")
```

```
with col1:
```

```
gerar_clicked = st.button(" Gerar Cartões", use_container_width=True,  
disabled=rows_cartao.count == 0)
```

```
with col2:
```

```
cbf_clicked = st.button(" Cartões CBF", use_container_width=True,disabled=rows_cartao.count !=  
0)
```

```
with col3:
```

```
add_clicked = st.button(" Incluir", type="primary", use_container_width=True,disabled=not  
selecionados.empty)
```

```
with col4:
```

```
edit_clicked = st.button(" Alterar", use_container_width=True,disabled=len(selecionados) != 1)
```

```
with col5:
```

```
delete_clicked = st.button(" Excluir", use_container_width=True,disabled=selecionados.empty)
```


IA:

The user wants to fix the button enable/disable logic:

- "Gerar Cartões" button should be ENABLED when rows_cartao has 0 items (empty list), DISABLED otherwise
- "Cartões CBF" button should be ENABLED when rows_cartao != 0 (has items), DISABLED otherwise

Their current code has a bug:

```
```python
```

```
disabled=rows_cartao.count == 0
```

```
```
```

rows_cartao.count is a method reference (list.count method), not a call. So rows_cartao.count == 0 compares a method object with 0, which is always False. So the button is never disabled via that logic.

The fix:

- rows_cartao is a list, so use len(rows_cartao) to get the count
- "Gerar Cartões": enabled when list is empty !' disabled=len(rows_cartao) != 0 or equivalently disabled=len(rows_cartao) > 0
- "Cartões CBF": enabled when list is not empty !' disabled=len(rows_cartao) == 0

Let me write the corrected code:

```
```python
```

```
col1, col2, col3, col4, col5 = st.columns(5, gap="small")
```

```
with col1:
```

```
gerar_clicked = st.button(
```

```
" Gerar Cartões",
```

```
use_container_width=True,
```

```
disabled=len(rows_cartao) > 0 # habilitado apenas quando NÃO há cartões
```

```
)
```

with col2:

```
cbf_clicked = st.button(
 " Cartões CBF",
 use_container_width=True,
 disabled=len(rows_cartao) == 0 # habilitado apenas quando há cartões
)
'''
```

Explanation of the bug:

- `rows_cartao.count` without parentheses is a reference to the list's count METHOD, not the count itself
- Comparing a method object with `== 0` is always False
- To get the number of elements: `len(rows_cartao)`

Also note that `rows_cartao.count()` requires an argument (the value to count), so `rows_cartao.count` was definitely not intended. `len()` is the correct way.

I should respond in Portuguese since the user writes in Portuguese. Keep it concise and clear. O problema é que `rows_cartao.count` **não é o número de elementos** — é uma referência ao *método* count da lista (sem os parênteses, o Python não o executa). Comparar um método com `== 0` sempre retorna False, então os botões nunca são desabilitados pela sua condição.

Para contar elementos de uma lista, use `len()`.

## Código corrigido

```
```python
col1, col2, col3, col4, col5 = st.columns(5, gap="small")

with col1:

    gerar_clicked = st.button(
        " Gerar Cartões",
```

```
use_container_width=True,

disabled=len(rows_cartao) > 0 # habilitado apenas quando NÃO há cartões

)

with col2:

    cbf_clicked = st.button(

        " Cartões CBF",

        use_container_width=True,

        disabled=len(rows_cartao) == 0 # habilitado apenas quando HÁ cartões

    )

    with col3:

        add_clicked = st.button(

            " Incluir",

            type="primary",

            use_container_width=True,

            disabled=not selecionados.empty

        )

        with col4:

            edit_clicked = st.button(

                " Alterar",

                use_container_width=True,

                disabled=len(selecionados) != 1

            )

            with col5:

                delete_clicked = st.button(

                    " Excluir",
```

```
use_container_width=True,  
  
disabled=selecionados.empty  
  
)  
  
...
```

Lógica aplicada

Botão	Habilitado quando	Desabilitado quando
Gerar Cartões	<code>len(rows_cartao) == 0</code> (nenhum cartão cadastrado)	Já existem cartões
Cartões CBF	<code>len(rows_cartao) > 0</code> (há cartões cadastrados)	Nenhum cartão cadastrado

> **Observação:** o fluxo faz sentido — "Gerar Cartões" cria os registros iniciais (só faz sentido quando não existem), e "Cartões CBF" atualiza/atualiza os existentes (só faz sentido quando já existem). A condição de um é o inverso exato do outro.

Usuário:

A sugestão não deu certo.

O botão "Gerar Cartões" ficou habilitado para 2026, que tem registros, e ficou habilitado para 2027, que não tem registros.

E para "Cartões CBF" ficou desabilitado para 2026 e 2027, o correto é ficar desabilitado para 2027 pois não possui registros e habilitado para 2026 que possui registros

IA:

O problema é que `rows_cartao` pode estar vazio mesmo quando deveria ter dados (por causa de cache do `@st.cache_data` ou erro silencioso na API). A solução é usar o **DataFrame diretamente** — `cartaoDF` é o objeto que está sendo exibido na tela, então é a fonte mais confiável.

Código corrigido

Substitua o bloco dos botões por:

```
```python

col1, col2, col3, col4, col5 = st.columns(5, gap="small")

with col1:

 gerar_clicked = st.button(

 " Gerar Cartões",

 use_container_width=True,

 disabled=not cartaoDF.empty # habilitado quando DF está vazio

)

with col2:

 cbf_clicked = st.button(

 " Cartões CBF",

 use_container_width=True,

 disabled=cartaoDF.empty # habilitado quando DF tem dados

)

with col3:

 add_clicked = st.button(

 " Incluir",

 type="primary",

 use_container_width=True,

 disabled=not selecionados.empty

)

with col4:
```

```

edit_clicked = st.button(

" Alterar",

use_container_width=True,

disabled=len(selecionados) != 1

)

with col5:

delete_clicked = st.button(

" Excluir",

use_container_width=True,

disabled=selecionados.empty

)

...

```

## Por que `cartaoDF.empty` é mais confiável que `len(rows_cartao)`

Verificação	Problema potencial
<code>len(rows_cartao)</code>	Se a API falhar silenciosamente ou o cache retornar lista vazia, <code>rows_cartao</code> fica <code>[]</code> mesmo havendo dados no banco
<code>cartaoDF.empty</code>	O DataFrame é construído depois da API e é o mesmo objeto exibido na tela — se há dados na tela, <code>cartaoDF.empty</code> é <code>False</code>

## Lógica aplicada

Cenário	cartaoDF.empty	Gerar Cartões	Cartões CBF
2026 (tem registros na tela)	False	disabled=not False = desabilitado	disabled=False = habilitado
2027 (DF vazio na tela)	True	disabled=not True = habilitado	disabled=True = desabilitado

> Se mesmo com `cartaoDF.empty` o problema persistir, adicione temporariamente `st.write(f'cartaoDF.empty= {cartaoDF.empty}')` logo acima dos botões para verificar o valor real em tempo de execução.

## Usuário:

No projeto do Brasileirao após o termino de cada rodada, clacula-se a classificação do Brasileirão e em seguida tenho um endpoint para salvar a classificação da rodada, portanto para cada rodada tenho um retrato da classificação da rodada, ou seja, se estou na rodada 20 então tenho 20 classificações.

Tenho um `classificacao_rodada_dto.py` :

```
from pydantic import BaseModel
```

```
from typing import Optional, List
```

```
class ClassificacaoRodadaBaseSchema(BaseModel):
```

```
.....
```

Schema base para classificação por rodada.

Inclui informações essenciais como série, ano, rodada, pontos, vitórias, saldo de gols, gols pró, confronto direto e sigla do clube.

```
.....
```

```
clr_id: int
```

```
clr_serie: str
```

```
clr_ano: int
```

```
clr_rodada: Optional[str] = None
```

clr\_pontos: int

clr\_vitorias: int

clr\_saldo\_gols: int

clr\_gols\_pro: int

clr\_confronto\_direto: int

clube\_clu\_sigla: str

clr\_qtd\_jogou: Optional[int] = None

clr\_qtd\_empates: Optional[int] = None

clr\_qtd\_derrotas: Optional[int] = None

clr\_gols\_contra: Optional[int] = None

A tabela que armazena é a classificacao\_rodada definida em:

```
from typing import TYPE_CHECKING, Optional
```

```
from sqlalchemy import String, Integer, ForeignKey, DateTime, text, Identity, func
```

```
from sqlalchemy.orm import Mapped, mapped_column, relationship
```

```
from app.core.database import Base
```

```
if TYPE_CHECKING:
```

```
from app.models.clube_models import Clube
```

```
class ClassificacaoRodada(Base):
```

```
 __tablename__ = "classificacao_rodada"
```

```
 clr_id: Mapped[int] = mapped_column(
```

```
 Integer,
```

```
 Identity(),
```

```
 primary_key=True,
```

```
 autoincrement=True
```

```
) # ID da classificação
```



```
clr_serie: Mapped[str] = mapped_column(
```

```
String(1),
```

```
nullable=False
```

```
) # Série do campeonato (A ou B)
```

```
clr_ano: Mapped[int] = mapped_column(
```

```
Integer,
```

```
nullable=False
```

```
) # Ano da competição
```

```
clr_rodada: Mapped[Optional[str]] = mapped_column(
```

```
String(45),
```

```
nullable=True
```

```
) # Número ou descrição da rodada
```

```
clr_pontos: Mapped[int] = mapped_column(
```

```
Integer,
```

```
nullable=False
```

```
) # Pontos do clube
```

```
clr_vitorias: Mapped[int] = mapped_column(
```

```
Integer,
```

```
nullable=False
```

```
) # Número de vitórias
```

```
clr_saldo_gols: Mapped[int] = mapped_column(
```

```
Integer,
```

```
nullable=False
```

```
) # Saldo de gols
```

```
clr_gols_pro: Mapped[int] = mapped_column(
 Integer,
 nullable=False
) # Gols marcados ("gols pró")

clr_confronto_direto: Mapped[int] = mapped_column(
 Integer,
 nullable=False
) # Critério de confronto direto

clube_clu_sigla: Mapped[str] = mapped_column(
 String(3),
 ForeignKey("clube.clu_sigla"),
 nullable=False
) # FK do clube

clr_qtd_jogou: Mapped[Optional[int]] = mapped_column(
 Integer,
 nullable=True
) # Quantidade de jogos

clr_qtd_empates: Mapped[Optional[int]] = mapped_column(
 Integer,
 nullable=True
) # Quantidade de empates

clr_qtd_derrotas: Mapped[Optional[int]] = mapped_column(
 Integer,
 nullable=True
) # Quantidade de derrotas
```

```
clr_gols_contra: Mapped[Optional[int]] = mapped_column(
 Integer,
 nullable=True
) # Gols sofridos ("gols contra")
```

## Relacionamento com a tabela Clube

```
clube: Mapped["Clube"] = relationship(
 "Clube",
 foreign_keys=[clube_clu_sigla]
)

def __init__(self, clr_serie, clr_ano, clr_rodada, clr_pontos, clr_vitorias, clr_saldo_gols,
 clr_gols_pro, clr_confronto_direto, clube_clu_sigla, clr_qtd_jogou=None, clr_qtd_empates=None,
 clr_qtd_derrotas=None, clr_gols_contra=None):
 self.clr_serie = clr_serie
 self.clr_ano = clr_ano
 self.clr_rodada = clr_rodada
 self.clr_pontos = clr_pontos
 self.clr_vitorias = clr_vitorias
 self.clr_saldo_gols = clr_saldo_gols
 self.clr_gols_pro = clr_gols_pro
 self.clr_confronto_direto = clr_confronto_direto
 self.clube_clu_sigla = clube_clu_sigla
 self.clr_qtd_jogou = clr_qtd_jogou
 self.clr_qtd_empates = clr_qtd_empates
 self.clr_qtd_derrotas = clr_qtd_derrotas
```

```
self.clr_gols_contra = clr_gols_contra

def as_dict(self):

 return {

 "clr_id": self.clr_id,

 "clr_serie": self.clr_serie,

 "clr_ano": self.clr_ano,

 "clr_rodada": self.clr_rodada,

 "clr_pontos": self.clr_pontos,

 "clr_vitorias": self.clr_vitorias,

 "clr_saldo_gols": self.clr_saldo_gols,

 "clr_gols_pro": self.clr_gols_pro,

 "clr_confronto_direto": self.clr_confronto_direto,

 "clube_clu_sigla": self.clube_clu_sigla,

 "clr_qtd_jogou": self.clr_qtd_jogou,

 "clr_qtd_empates": self.clr_qtd_empates,

 "clr_qtd_derrotas": self.clr_qtd_derrotas,

 "clr_gols_contra": self.clr_gols_contra

 }
```

Com base ClassificacaoRodadaBaseSchema sugira alguns gráficos e para cada gráfico sugira um schema para o gráfico, quero lembrar que a posição na classificação será gerado ao executar a query do gráfico.

Monte a query base base no modelo.

## IA:

O usuário está pedindo sugestões de gráficos para visualização da classificação por rodada do Brasileirão. Ele tem:

1. ClassificacaoRodadaBaseSchema - DTO com campos: clr\_id, clr\_serie, clr\_ano, clr\_rodada, clr\_pontos, clr\_vitorias, clr\_saldo\_gols, clr\_gols\_pro, clr\_confronto\_direto, clube\_clu\_sigla, clr\_qtd\_jogou, clr\_qtd\_empates, clr\_qtd\_derrotas, clr\_gols\_contra

2. Tabela classificacao\_rodada no SQLAlchemy com relacionamento com Clube

Ele quer:

- Sugestões de gráficos
- Para cada gráfico, um schema Pydantic
- A posição na classificação será gerada ao executar a query (não é armazenada)
- Montar a query base no modelo (SQLAlchemy)

Vou pensar nos gráficos clássicos de campeonato de futebol:

1. **Evolução da classificação por rodada (linha)** - posição de cada clube ao longo das rodadas - gráfico de linhas clássico "corrida pelo título"
2. **Evolução de pontos por rodada** - linha dos pontos acumulados
3. **Classificação atual (barra)** - pontos dos clubes na rodada X
4. **Top N em determinada rodada** - barra horizontal
5. **Gols pró vs gols contra (scatter/barras)** - ataque vs defesa
6. **Aproveitamento de pontos (%)** - pontos ganhos / pontos disputados
7. **Movimentação (subiu/caiu)** - comparação entre duas rodadas

O usuário pediu "sugira alguns gráficos e para cada gráfico sugira um schema para o gráfico" - ou seja, schemas Pydantic para a resposta da API de cada gráfico.

Também: "Monte a query base no modelo" - query SQLAlchemy que calcula a posição (rank) via window function.

Vou montar:

## Query base com posição

A query base calcula a posição usando window functions (ROW\_NUMBER ou RANK) sobre os critérios do Brasileirão:

1. Pontos (desc)
2. Vitórias (desc)

3. Saldo de gols (desc)

4. Gols pró (desc)

5. Confronto direto

O critério de desempate do Brasileirão: 1º pontos, 2º vitórias, 3º saldo de gols, 4º gols pró, depois confronto direto. Na verdade a ordem oficial é: pontos ganhos, vitórias, saldo de gols, gols pró, confronto direto (confronto direto entre dois clubes específicos é complexo em SQL, então normalmente se usa como critério desempate adicional).

```
```python
```

```
from sqlalchemy import select, func
```

```
from app.models.classificacao_rodada_models import ClassificacaoRodada
```

```
from app.models.clube_models import Clube
```

```
from sqlalchemy.orm import aliased
```

```
query_base = (
```

```
select(
```

```
ClassificacaoRodada.clr_rodada,
```

```
ClassificacaoRodada.clube_clu_sigla,
```

```
Clube.clu_nome,
```

```
Clube.clu_link_escudo,
```

```
ClassificacaoRodada.clr_pontos,
```

```
ClassificacaoRodada.clr_vitorias,
```

```
ClassificacaoRodada.clr_saldo_gols,
```

```
ClassificacaoRodada.clr_gols_pro,
```

```
ClassificacaoRodada.clr_gols_contra,
```

```
ClassificacaoRodada.clr_qtd_jogou,
```

```
ClassificacaoRodada.clr_qtd_empates,
```

```
ClassificacaoRodada.clr_qtd_derrotas,
```

```
func.row_number().over(
```

```

order_by=[
    ClassificacaoRodada.clr_pontos.desc(),
    ClassificacaoRodada.clr_vitorias.desc(),
    ClassificacaoRodada.clr_saldo_gols.desc(),
    ClassificacaoRodada.clr_gols_pro.desc(),
],
partition_by=ClassificacaoRodada.clr_rodada
).label("posicao")
)
.join(Clube, Clube.clu_sigla == ClassificacaoRodada.clube_clu_sigla)
.where(
    ClassificacaoRodada.clr_serie == serie,
    ClassificacaoRodada.clr_ano == ano
)
)
...

```

Gráficos sugeridos

1. Evolução da Posição (Gráfico de Linha — "A Corrida do Campeonato")

- Eixo X: rodadas
- Eixo Y: posição (invertida, 1 no topo)
- Uma linha por clube

Schema:

```
```python
```

```
class PosicaoPorRodadaSchema(BaseModel):
```

```
 clube_clu_sigla: str
```

```
clu_nome: str

clu_link_escudo: Optional[str] = None

posicoes: List[int] # posição em cada rodada

rodadas: List[int]

...


```

Ou formato tabular (melhor para DataFrame no Streamlit):

```
```python

class ClassificacaoLinhaSchema(BaseModel):

    clr_rodada: int

    clube_clu_sigla: str

    clu_nome: str

    clu_link_escudo: Optional[str] = None

    posicao: int

    clr_pontos: int

...


```

2. Evolução de Pontos Acumulados (Linha)

3. Classificação de uma rodada específica (Barra Horizontal)

- Clubes no eixo Y, pontos no eixo X, ordenado
- Com escudo do clube

4. Ataque vs Defesa (Barras duplas ou dispersão)

- Gols pró vs gols contra

5. Aproveitamento (%) — barra horizontal

- pontos / (jogos - 3) 100

6. Comparativo de desempenho entre clubes (radar ou barras agrupadas)

- vitórias, empates, derrotas

Vou estruturar a resposta com:

1. Query base (SQLAlchemy com ROW_NUMBER para gerar posição)
2. Gráficos sugeridos com schema e query específica para cada

O usuário escreve em português, então respondo em português.

O contexto é um projeto FastAPI backend + Streamlit frontend, com a skill de Arquitetura FastAPI Enterprise já carregada. Devo seguir o padrão enterprise: DTOs no formato correto, services, etc.

Vou organizar:

Query base (posição calculada)

Gráfico 1: Evolução da Posição por Rodada (linha)

- schema: ClassificacaoPosicaoSchema (formato "long" - uma linha por clube/rodada)
- frontend faz pivot

Gráfico 2: Classificação da Rodada (barra horizontal)

- schema: ClassificacaoSnapshotSchema

Gráfico 3: Evolução de Pontos (linha)

Gráfico 4: Ataque vs Defesa (dispersão ou barras)

Gráfico 5: Aproveitamento (barra)

Gráfico 6: Performance V/E/D (barras empilhadas)

Para cada um, mostrar schema + query.

Vou ser conciso mas completo. Usar código nos blocos.

Detalhe importante: `clr_rodada` é `String(45)` no modelo (opcional). Isso é estranho para ordenação - precisaria cast para `int` na ordenação. Vou mencionar isso.

Vou escrever a resposta em português com markdown bem estruturado. Com base no seu modelo, a posição não está armazenada — ela deve ser calculada com **window function** (`ROW_NUMBER`) aplicando os critérios de desempate do Brasileirão: **pontos !' vitórias !' saldo de gols !' gols pró !' confronto direto**.

Query base (gera a posição)

```
```python
```

```
from sqlalchemy import select, func, cast, Integer
```

```
from app.models.classificacao_rodada_models import ClassificacaoRodada
```

```
from app.models.clube_models import Clube
```

```
def query_classificacao_base(serie: str, ano: int, rodada: int = None):
```

```
 """
```

Query base com a POSIÇÃO calculada via window function.

Filtra por série/ano e opcionalmente por rodada.

```
 """
```

```
 posicao = func.row_number().over(
```

```
 partition_by=cast(ClassificacaoRodada.clr_rodada, Integer),
```

```
 order_by=[
```

```
 ClassificacaoRodada.clr_pontos.desc(),
```

```
 ClassificacaoRodada.clr_vitorias.desc(),
```

```
 ClassificacaoRodada.clr_saldo_gols.desc(),
```

```
 ClassificacaoRodada.clr_gols_pro.desc(),
```

```
 ClassificacaoRodada.clr_confronto_direto.desc(),
```

```
],
```

```

).label("posicao")

query = (
 select(
 posicao,
 cast(ClassificacaoRodada.clr_rodada, Integer).label("clr_rodada"),
 ClassificacaoRodada.clube_clu_sigla,
 Clube.clu_nome.label("clu_nome"),
 Clube.clu_link_escudo.label("clu_link_escudo"),
 ClassificacaoRodada.clr_pontos,
 ClassificacaoRodada.clr_vitorias,
 ClassificacaoRodada.clr_saldo_gols,
 ClassificacaoRodada.clr_gols_pro,
 ClassificacaoRodada.clr_gols_contra,
 ClassificacaoRodada.clr_qtd_jogou,
 ClassificacaoRodada.clr_qtd_empates,
 ClassificacaoRodada.clr_qtd_derrotas,
)
 .join(Clube, Clube.clu_sigla == ClassificacaoRodada.clube_clu_sigla)
 .where(
 ClassificacaoRodada.clr_serie == serie,
 ClassificacaoRodada.clr_ano == ano,
)
)

if rodada is not None:
 query = query.where(cast(ClassificacaoRodada.clr_rodada,Integer) == rodada)

```

```
return query
```

```
...
```

> **Nota:** `clr_rodada` é `String(45)` no seu modelo, por isso o `cast(..., Integer)` — sem ele, a ordenação seria lexicográfica ("`10`" < "`2`"). Vale considerar migrar a coluna para `Integer` no futuro.

## Gráficos sugeridos

### 1. Evolução da Posição por Rodada ("Corrida do Campeonato")

O clássico gráfico de linha — eixo X = rodadas, eixo Y = posição (invertida, líder no topo). Uma linha por clube.

```
```python
```

```
class ClassificacaoEvolucaoSchema(BaseModel):
```

```
    """Uma linha por clube/rodada — o frontend faz o pivot para o gráfico de linha."""
```

```
    clr_rodada: int
```

```
    posicao: int
```

```
    clube_clu_sigla: str
```

```
    clu_nome: str
```

```
    clu_link_escudo: Optional[str] = None
```

```
    clr_pontos: int
```

```
...
```

```
```python
```

**Query: `query_classificacao_base(serie, ano)` —  
TODAS as rodadas**

# Service retorna List [ClassificacaoEvolucaoSchema]

...

No Streamlit: pivot com `pd.pivot_table(df, index="clr_rodada", columns="clu_nome", values="posicao")` e `st.line_chart` com eixo Y invertido.

## 2. Classificação de uma Rodada Específica (Barra Horizontal)

Retrato da tabela em uma rodada — barras horizontais com pontos, escudo ao lado.

```
```python

class ClassificacaoSnapshotSchema(BaseModel):

    """Snapshot da classificação em uma rodada específica."""

    posicao: int

    clube_clu_sigla: str

    clu_nome: str

    clu_link_escudo: Optional[str] = None

    clr_pontos: int

    clr_vitorias: int

    clr_qtd_jogou: Optional[int] = None

    clr_qtd_empates: Optional[int] = None

    clr_qtd_derrotas: Optional[int] = None

    clr_saldo_gols: int

    clr_gols_pro: int

    clr_gols_contra: Optional[int] = None

...

```python
```

**Query: query\_classificacao\_base(serie, ano, rodada=rodada) ordenada por posição**

...

### **3. Evolução de Pontos Acumulados (Linha)**

Mesmo formato do gráfico 1, mas plotando clr\_pontos no eixo Y — mostra o ritmo de pontuação.

```
```python
```

```
class PontosEvolucaoSchema(BaseModel):
```

```
    clr_rodada: int
```

```
    clube_clu_sigla: str
```

```
    clu_nome: str
```

```
    clu_link_escudo: Optional[str] = None
```

```
    clr_pontos: int
```

```
    posicao: int
```

```
...
```

```
```python
```

**Query: mesma query base (todas as rodadas), mesmo schema do gráfico 1 serve.**

**A diferença é apenas qual campo o frontend plota (pontos vs posição).**

...

> Os gráficos 1 e 3 podem compartilhar o mesmo endpoint e schema — ClassificacaoEvolucaoSchema já traz posicao e clr\_pontos.

## 4. Ataque vs Defesa (Dispersão ou Barras Duplas)

Gols pró × gols contra — quadrante superior esquerdo = melhor ataque e defesa.

```
```python

class AtaqueDefesaSchema(BaseModel):

    clube_clu_sigla: str

    clu_nome: str

    clu_link_escudo: Optional[str] = None

    posicao: int

    clr_gols_pro: int

    clr_gols_contra: int

    clr_saldo_gols: int

    ...

```python
```

**Query: query\_classificacao\_base(serie, ano, rodada=rodada)**

```
...
```

## 5. Aproveitamento de Pontos (Barra Horizontal)

Percentual de pontos conquistados sobre os disputados — normaliza clubes com jogos diferentes (útil com rodadas atrasadas).

```
```python

class AproveitamentoSchema(BaseModel):

    posicao: int

    clube_clu_sigla: str
```

```

clu_nome: str

clu_link_escudo: Optional[str] = None

clr_pontos: int

clr_qtd_jogou: int

aproveitamento: float # calculado na query: pontos / (jogos - 3) * 100
...

```python

from sqlalchemy import case

query = query_classificacao_base(serie, ano, rodada=rodada).add_columns(

case(

(

ClassificacaoRodada.clr_qtd_jogou > 0,

func.round(

cast(ClassificacaoRodada.clr_pontos, Integer) * 100.0

/ (ClassificacaoRodada.clr_qtd_jogou * 3),

1,

),

),

else_=0.0,

).label("aproveitamento")

)
...

```

## 6. Distribuição V / E / D (Barras Empilhadas)



Composição do desempenho — barras empilhadas por clube.

```
```python
class ResultadoDistribuicaoSchema(BaseModel):

    posicao: int

    clube_clu_sigla: str

    clu_nome: str

    clu_link_escudo: Optional[str] = None

    clr_qtd_jogou: int

    vitórias: int

    empates: int

    derrotas: int
...

```python
```

**Query: query\_classificacao\_base(serie, ano, rodada=rodada)**

**O service mapeia clr\_vitorias !' vitórias,  
clr\_qtd\_empates !' empates,**

**clr\_qtd\_derrotas !' derrotas**

...

## Resumo

#	Gráfico	Tipo	Schema	Query
---	---------	------	--------	-------

1	Evolução da posição	Linha	ClassificacaoEvolucaoSchema	Base, todas as rodadas
2	Classificação da rodada	Barra horizontal	ClassificacaoSnapshotSchema	Base, rodada=X
3	Evolução de pontos	Linha	ClassificacaoEvolucaoSchema (reusa)	Base, todas as rodadas
4	Ataque vs Defesa	Dispersão	AtaqueDefesaSchema	Base, rodada=X
5	Aproveitamento %	Barra horizontal	AproveitamentoSchema	Base + coluna calculada
6	V / E / D	Empilhada	ResultadoDistribuicaoSchema	Base, rodada=X

Todos partem da **mesma query base** — a diferenciação é apenas filtro (rodada específica ou não) e colunas calculadas. No backend, isso vira **um único repository method** ( `get_classificacao(serie, ano, rodada=None)`) e a camada de service/DTO adapta o resultado por endpoint — alinhado com a separação repository/service/DTO do padrão enterprise que você já usa.